```
// ===== FILE: apps/desktop/src-tauri/src/lib.rs =====
mod app_info;
mod asr_setup;
mod asr_sidecar;
mod blocking_http;
mod bundled_asr_assets;
mod command_error;
mod db;
mod diagnostic;
mod diagnostic_db_sanitize;
mod export_docx;
mod export_docx_polish_track;
mod local_asr_language;
mod local_asr_model;
mod local_runtime;
mod media_base_dir;
mod media_base_relink;
mod media_base_relocate;
mod native_audio;
mod online_stt_bridge;
mod packaged_hints;
mod postprocess_cmd;
mod profile;
mod project;
mod secret_keyring_session;
mod secret_store_policy;
mod stt_native;
mod stt_online_api_key_cmd;
mod stt_online_probe;
mod stt_online_secret_store;
mod utils;

use std::path::PathBuf;
use std::sync::Arc;

use tauri::{Manager, RunEvent};

use crate::db::DbPool;

#[derive(Clone)]
pub struct DbState {
    pub root: PathBuf,
    pub db_path: PathBuf,
    pool: Arc<DbPool>,
}

impl DbState {
    pub fn new(root: PathBuf, db_path: PathBuf, pool: DbPool) -> Self {
        Self {
            root,
```

```
        db_path,
        pool: Arc::new(pool),
      }
    }

    pub(crate) fn pool(&self) -> &DbPool {
      &self.pool
    }

    #[cfg(test)]
    pub fn open_test_db(root: PathBuf) -> Self {
      Self::open_test_db_at(root.clone(), root.join("rushi.sqlite3"))
    }

    #[cfg(test)]
    pub fn open_test_db_at(root: PathBuf, db_path: PathBuf) -> Self {
      let pool = crate::db::bootstrap_db_at(&db_path).expect("test db bootstrap");
      Self::new(root, db_path, pool)
    }
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    let app = tauri::Builder::default()
      .plugin(tauri_plugin_process::init())
      .plugin(tauri_plugin_updater::Builder::new().build())
      .setup(|app| {
        bundled_asr_assets::init_from_app(app.handle());
        let st = project::setup_db(app.handle())?;
        let build_info = app_info::build_app_build_info(Some(&st.root), Some(&st.db_path));
        project::utils::append_desktop_log_line(
          &st,
          &format!(
            "INFO parity startup: profile={} platform={}-{}",
            build_info.shell_profile, build_info.platform_os, build_info.platform_arch
          ),
        );
        project::utils::append_desktop_log_line(
          &st,
          &format!(
            "INFO parity bundle: asr_shell_managed={} bundled_sidecar_build={}",
            build_info.asr_shell_managed,
            if build_info.bundled_sidecar_build.is_some() {
              "present"
            } else {
              "missing"
            }
          ),
        );
        let automation_diag_state = st.clone();
```

```
    app.manage(st);
    #[cfg(feature = "devtools")]
    if std::env::var_os("RUSHI_DEVTOOLS").is_some() {
      if let Some(w) = app.get_webview_window("main") {
        w.open_devtools();
      }
    }
    app.manage(asr_sidecar::AsrSidecarState(std::sync::Mutex::new(None)));
    app.manage(asr_sidecar::BundledAsrLaunchState(std::sync::Mutex::new(
      asr_sidecar::BundledAsrLaunchReport::default(),
    )));
    app.manage(asr_sidecar::AsrSupervisorState(std::sync::Mutex::new(
      asr_sidecar::supervisor::SupervisorSnapshot::new_session(),
    )));
    app.manage(local_runtime::installer::LocalRuntimeInstallerState::default());
    app.manage(asr_sidecar::cuda_install::AsrCudaInstallerState::default());
    app.manage(postprocess_cmd::PostprocessCancelState::default());
    app.manage(project::TranscribeCancelState::default());
    app.manage(native_audio::NativeAudioState::default());
    if std::env::var_os("RUSHI_AUTOMATION").as_deref() == Some(std::ffi::OsStr::new("1"))
      && std::env::var_os("RUSHI_AUTOMATION_DIAGNOSTIC_ZIP").is_some()
    {
      let path = std::env::var_os("RUSHI_AUTOMATION_DIAGNOSTIC_ZIP").unwrap();
      let diag_handle = app.handle().clone();
      let diag_state = automation_diag_state.clone();
      let diag_path = PathBuf::from(path);
      tauri::async_runtime::spawn(async move {
        tokio::time::sleep(std::time::Duration::from_secs(20)).await;
        let diag_state_for_log = diag_state.clone();
        let diag_path_for_log = diag_path.clone();
        let result = tauri::async_runtime::spawn_blocking(move || {
          diagnostic::write_diagnostic_bundle_to_path(
            &diag_handle,
            &diag_state,
            &diag_path,
          )
        })
        .await
        .unwrap_or_else(|e| Err(format!("diagnostic task join failed: {e}")));
        let line = match result {
          Ok(()) => format!(
            "INFO parity startup: automation_diagnostic_zip=ok path={}",
            diag_path_for_log.display()
          ),
          Err(e) => format!(
            "WARN parity startup: automation_diagnostic_zip=failed err={}",
            e.replace(['\n', '\r'], " ")
          ),
        };
        project::utils::append_desktop_log_line(&diag_state_for_log, &line);
```

```
        });
    }
    let handle = app.handle().clone();
    let start_handle = handle.clone();
    tauri::async_runtime::spawn(async move {
      let _ = tauri::async_runtime::spawn_blocking(move || {
        asr_sidecar::try_start_bundled(&start_handle);
      })
      .await;
      asr_sidecar::warm::spawn_watchdog(handle);
    });
    #[cfg(target_os = "macos")]
    app_info::attach_macos_app_menu(app)?;
    Ok(())
  })
  .invoke_handler(tauri::generate_handler![
    app_info::app_version,
    app_info::app_build_info,
    app_info::read_third_party_licenses,
    app_info::open_bundled_user_guide,
    asr_setup::diagnose::asr_setup_diagnose,
    asr_sidecar::bundled::launch::bundled_asr_launch_report,
    // Intentional defer: registered for env-page FSM; no TS invoke yet (r3h-i1).
    asr_sidecar::supervisor::asr_supervisor_snapshot,
    local_runtime::local_runtime_diagnose,
    local_runtime::installer::commands::local_runtime_download_sidecar,
    local_runtime::installer::commands::local_runtime_cancel_download,
    local_runtime::recovery::commands::revalidate::local_runtime_revalidate_install,
    local_runtime::recovery::commands::clear::local_runtime_clear_install,
    local_runtime::recovery::commands::restore::local_runtime_restore_previous,
    asr_sidecar::cuda_install::commands::asr_cuda_sidecar_status,
    asr_sidecar::cuda_install::commands::asr_download_cuda_sidecar,
    asr_sidecar::cuda_install::commands::asr_cancel_cuda_sidecar_download,
    project::picker_cmd::pick_audio_path,
    project::picker_cmd::pick_audio_paths,
    project::project_create_cmd::project_create_from_audio,
    project::project_create_cmd::create_empty_project,
    project::project_create_cmd::create_project_from_text,
    project::project_create_cmd::create_empty_text_file,
    project::project_create_cmd::import_audio_to_project,
    project::project_create_cmd::import_text_to_project,
    project::file_import_cmd::import_transcript_to_project,
    project::import_duplicate::check_project_import_duplicate,
    project::project_query_cmd::project_list,
    project::project_query_cmd::project_load,
    project::project_query_cmd::project_list_edit_log,
    project::project_query_cmd::project_record_edit_log,
    project::segment_cmd::file_save_segments,
    project::speaker_cmd::file_create_speaker,
    project::speaker_cmd::file_rename_speaker,
```

project::speaker_cmd::file_reassign_segments_speaker,
project::segment_cmd::file_restore_segments_from_edit_log,
project::run_transcribe_cmd::sync::project_run_transcribe,
project::transcribe_cancel_cmd::project_cancel_transcribe,
project::run_transcribe_cmd::async_cmd::project_transcribe_async_start,
project::run_transcribe_cmd::async_cmd::project_transcribe_async_finalize,
project::run_transcribe_cmd::sync::get_last_transcribe_timeline,
project::run_transcribe_cmd::sync::record_transcribe_timeline_poll_progress,
project::run_transcribe_cmd::sync::record_transcribe_timeline_poll_failure,
project::project_delete_cmd::project_delete,
project::project_metadata_cmd::rename_project,
project::project_metadata_cmd::update_project_metadata,
project::picker_cmd::pick_text_path,
project::picker_cmd::ui_desktop_log,
project::file_cmd::list_files,
project::file_cmd::load_file,
project::welcome_search_cmd::welcome_search_files,
project::welcome_search_cmd::welcome_search_content,
project::file_cmd::rename_file,
project::file_cmd::delete_file,
project::file_cmd::move_file_to_project,
project::file_cmd::copy_file_to_project,
project::file_cmd::reveal_project_in_file_manager,
project::file_cmd::reveal_file_in_file_manager,
project::waveform_peaks_cmd::ensure_waveform_peaks,
project::waveform_peaks_cmd::waveform_peaks_status,
project::waveform_asset_cmd::scoped_waveform_file_meta,
native_audio::commands::native_audio_load,
native_audio::commands::native_audio_play,
native_audio::commands::native_audio_pause,
native_audio::commands::native_audio_seek,
native_audio::commands::native_audio_set_rate,
native_audio::commands::native_audio_snapshot,
native_audio::commands::native_audio_stop,
project::export_cmd::export_project_bundle,
project::export_cmd::export_library_bundle,
project::export_cmd::import_project_bundle,
project::export_cmd::import_exchange_bundle_preview,
project::export_cmd::import_exchange_bundle_apply,
project::asr_runtime_paths_cmd::get_asr_runtime_paths,
project::asr_cache_cmd::asr_model_cache_info,
project::asr_cache_cmd::clear_asr_model_cache,
project::bundled_asr_models_seed::seed_bundled_asr_models_if_needed,
project::waveform_peaks_cache_cmd::waveform_peaks_cache_info,
project::waveform_peaks_cache_cmd::clear_orphan_waveform_peaks_cache,
project::waveform_peaks_cache_cmd::clear_waveform_peaks_for_file,
project::install_cmd::install_funasr_deps_interactive,
project::install_cmd::retry_bundled_asr_sidecar,
project::install_cmd::try_start_bundled_asr_sidecar,
asr_sidecar::loopback::asr_loopback_request,

asr_sidecar::asr_app_manages_bundled_sidecar,
asr_sidecar::kill_loopback_asr_listeners_cmd,
local_asr_model::get_local_asr_hub_model_pref,
local_asr_model::set_local_asr_hub_model_pref,
local_asr_language::get_local_asr_recognition_language_pref,
local_asr_language::set_local_asr_recognition_language_pref,
media_base_dir::get_media_base_dir_info,
media_base_dir::get_app_data_root_path,
media_base_dir::set_media_base_dir_pref,
media_base_dir::pick_media_base_dir,
media_base_relocate::get_media_base_managed_summary,
media_base_relocate::pick_media_base_dir_preview,
media_base_relocate::commit_media_base_dir_change,
project::export_cmd::open_app_data_folder,
project::export_cmd::export_text_file,
project::glossary_cmd::glossary_list,
project::glossary_cmd::glossary_add,
project::glossary_cmd::glossary_add_batch,
project::glossary_cmd::glossary_update,
project::glossary_cmd::glossary_delete_batch,
project::glossary_cmd::glossary_set_hotword_batch,
project::glossary_cmd::glossary_delete,
project::glossary_cmd::glossary_hotwords_preview,
project::glossary_cmd::glossary_import_from_file,
postprocess_cmd::postprocess_api_key_cmd::llm_save_api_key,
postprocess_cmd::postprocess_api_key_cmd::llm_delete_api_key,
postprocess_cmd::postprocess_api_key_cmd::llm_has_stored_api_key,
postprocess_cmd::postprocess_api_key_cmd::llm_migrate_legacy_api_key,
postprocess_cmd::postprocess_api_key_cmd::llm_probe_connection,
postprocess_cmd::postprocess_api_key_cmd::ollama_detect_status,
stt_online_probe::stt_probe_online_health,
stt_online_probe::stt_probe_xunfei_credentials,
stt_online_api_key_cmd::stt_save_api_key,
stt_online_api_key_cmd::stt_delete_api_key,
stt_online_api_key_cmd::stt_has_stored_api_key,
postprocess_cmd::postprocess_auto_punctuate_cmd::postprocess_auto_punctuate,
postprocess_cmd::postprocess_refine_cmd::postprocess_refine_segments,
postprocess_cmd::postprocess_stage_b_proofread_cmd::postprocess_stage_b_proofread,
postprocess_cmd::postprocess_prompt_defaults_cmd::get_llm_prompt_defaults,
postprocess_cmd::postprocess_cancel_cmd::postprocess_cancel_auto_punctuate,
postprocess_cmd::postprocess_cancel_cmd::postprocess_cancel_export_polish,
postprocess_cmd::postprocess_export_polish_cmd::postprocess_export_polish,
project::segment_cmd::correction_accept_rule,
project::segment_cmd::correction_stable_rules_list,
project::segment_cmd::correction_glossary_learn_prompts,
project::correction_memory_cmd::correction_memory_list,
project::correction_memory_cmd::correction_memory_save,
project::correction_memory_cmd::correction_memory_delete,
project::correction_memory_cmd::correction_glossary_mine_candidates,
project::lexicon_bundle_cmd::lexicon_bundle_export,

```
            project::lexicon_bundle_cmd::lexicon_bundle_export_preview,
            project::lexicon_bundle_cmd::lexicon_bundle_import_preview,
            project::lexicon_bundle_cmd::lexicon_bundle_import_apply,
            profile::export_settings_profile,
            profile::import_settings_profile,
            export_docx::export_docx,
            diagnostic::export_diagnostic_bundle,
            project::quality_eval::quality_get_last_report,
            project::quality_eval::quality_get_baseline_report,
            project::quality_eval::quality_run_eval,
            project::quality_eval::quality_import_report_file,
            project::quality_eval::quality_set_baseline_from_last,
            project::quality_eval::quality_last_report_path_cmd,
            project::quality_eval::quality_export_correction_memory_jsonl,
        ])
        .build(tauri::generate_context!())
        .expect("error while building tauri application");

    app.run(|app_handle, event| {
        if matches!(event, RunEvent::Exit) {
            native_audio::shutdown(app_handle);
            asr_sidecar::stop_bundled(app_handle);
        }
    });
}
// ===== FILE: apps/desktop/src-tauri/src/main.rs =====
// Prevents additional console window on Windows in release, DO NOT REMOVE!!
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

fn main() {
    rushi_desktop_lib::run();
}
// ===== FILE: apps/desktop/src-tauri/src/db/columns.rs =====
use rusqlite::Connection;

pub(crate) fn table_columns(conn: &Connection, table: &str) -> rusqlite::Result<Vec<String>> {
    if !matches!(
        table,
        "segments" | "files" | "projects" | "glossary_terms" | "edit_log_snapshots"
    ) {
        return Err(rusqlite::Error::InvalidParameterName(table.to_string()));
    }
    let mut stmt = conn.prepare(&format!("PRAGMA table_info({table})"))?;
    let names = stmt
        .query_map([], |row| row.get::<_, String>(1))?
        .collect::<Result<Vec<_>, _>>()?;
    Ok(names)
}
// ===== FILE: apps/desktop/src-tauri/src/db/migrations/correction_memory.rs =====
use rusqlite::Connection;
```

```
pub(super) fn migrate_correction_memory_p2(conn: &Connection) -> rusqlite::Result<()> {
  conn.execute_batch(
    r#"
    CREATE TABLE IF NOT EXISTS correction_memory (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      before_text TEXT NOT NULL,
      after_text TEXT NOT NULL,
      hit_count INTEGER NOT NULL DEFAULT 1,
      accepted_as_rule INTEGER NOT NULL DEFAULT 0,
      created_at_ms INTEGER NOT NULL,
      updated_at_ms INTEGER NOT NULL,
      UNIQUE (before_text, after_text)
    );
    CREATE INDEX IF NOT EXISTS idx_correction_memory_hits ON correction_memory(hit_count DESC...
    "#,
  )?;
  Ok(())
}
// ===== FILE: apps/desktop/src-tauri/src/db/migrations/edit_log.rs =====
use rusqlite::Connection;

use super::super::columns::table_columns;

pub(super) fn migrate_edit_log_snapshots(conn: &Connection) -> rusqlite::Result<()> {
  conn.execute_batch(
    r#"
    CREATE TABLE IF NOT EXISTS edit_log_snapshots (
      edit_log_id INTEGER PRIMARY KEY,
      file_id TEXT NOT NULL,
      segments_json TEXT NOT NULL,
      segment_count INTEGER NOT NULL,
      FOREIGN KEY (edit_log_id) REFERENCES edit_log(id) ON DELETE CASCADE
    );
    CREATE INDEX IF NOT EXISTS idx_edit_log_snapshots_file ON edit_log_snapshots(file_id);
    "#,
  )?;
  migrate_edit_log_snapshots_schema(conn)?;
  Ok(())
}

fn migrate_edit_log_snapshots_schema(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "edit_log_snapshots")?;
  if !cols.iter().any(|c| c == "schema_version") {
    conn.execute(
      "ALTER TABLE edit_log_snapshots ADD COLUMN schema_version INTEGER NOT NULL DEFAULT 1",
      [],
    )?;
  }
  Ok(())
```

```
}
// ===== FILE: apps/desktop/src-tauri/src/db/migrations/files.rs =====
use rusqlite::Connection;

use super::super::columns::table_columns;
use crate::project::file_name_unique::unique_file_name;

pub(super) fn migrate_files_import_provenance(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "files")?;
  if cols.is_empty() {
    return Ok(());
  }
  if !cols.iter().any(|c| c == "import_source_path") {
    conn.execute("ALTER TABLE files ADD COLUMN import_source_path TEXT", [])?;
  }
  if !cols.iter().any(|c| c == "import_content_sha256") {
    conn.execute(
      "ALTER TABLE files ADD COLUMN import_content_sha256 TEXT",
      [],
    )?;
  }
  if !cols.iter().any(|c| c == "import_source_size") {
    conn.execute(
      "ALTER TABLE files ADD COLUMN import_source_size INTEGER",
      [],
    )?;
  }
  if !cols.iter().any(|c| c == "import_source_modified_ms") {
    conn.execute(
      "ALTER TABLE files ADD COLUMN import_source_modified_ms INTEGER",
      [],
    )?;
  }
  // SHA256 backfill runs deferred after pool open (see bootstrap_db_at) so migrate never block...
  Ok(())
}

/// Hub list duration cache — written on import / open / transcribe probe.
pub(super) fn migrate_files_duration_sec(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "files")?;
  if cols.is_empty() {
    return Ok(());
  }
  if !cols.iter().any(|c| c == "duration_sec") {
    conn.execute("ALTER TABLE files ADD COLUMN duration_sec REAL", [])?;
  }
  Ok(())
}

/// R3 决策：`files.name` 加全局 UNIQUE 索引防并发写入产生重名（历史仅靠应用层
```

```
/// `unique_file_name` advisory 检查，存在竞态窗口）。若历史数据已有重名，迁移时
/// 保留每组中最早创建的一条，其余按 Finder 风格追加 ` (2)` 等后缀去重，并写入
/// `edit_log` 留痕，再建唯一索引。
pub(crate) fn migrate_files_name_unique(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "files")?;
  if cols.is_empty() {
    return Ok(());
  }
  let index_exists = conn.query_row(
    "SELECT COUNT(*) FROM sqlite_master WHERE type = 'index' AND name = 'idx_files_name_uniqu...
    [],
    |row| row.get::<_, i64>(0),
  )? > 0;
  if index_exists {
    return Ok(());
  }

  let mut dup_stmt = conn.prepare(
    "SELECT id, name, project_id FROM files WHERE name IN (
      SELECT name FROM files GROUP BY name HAVING COUNT(*) > 1
    ) ORDER BY name, created_at_ms ASC, id ASC",
  )?;
  let duplicates: Vec<(String, String, String)> = dup_stmt
    .query_map([], |row| {
      Ok((
        row.get::<_, String>(0)?,
        row.get::<_, String>(1)?,
        row.get::<_, String>(2)?,
      ))
    })?
    .collect::<Result<Vec<_>, _>>()?;
  drop(dup_stmt);

  let mut kept_names: std::collections::HashSet<String> = std::collections::HashSet::new();
  for (file_id, name, project_id) in duplicates {
    if kept_names.insert(name.clone()) {
      // Earliest-created row in this duplicate group keeps its name.
      continue;
    }
    let renamed = unique_file_name(conn, &name, Some(file_id.as_str())).map_err(|e| {
      rusqlite::Error::ToSqlConversionFailure(Box::new(std::io::Error::other(e)))
    })?;
    conn.execute(
      "UPDATE files SET name = ?1 WHERE id = ?2",
      rusqlite::params![renamed, file_id],
    )?;
    let detail = serde_json::json!({
      "file_id": file_id,
      "old_name": name,
      "new_name": renamed,
```

```
      })
      .to_string();
    conn.execute(
      "INSERT INTO edit_log (project_id, at_ms, kind, detail) VALUES (?1, ?2, ?3, ?4)",
      rusqlite::params![
        project_id,
        chrono::Utc::now().timestamp_millis(),
        "migration_dedupe_file_name",
        detail,
      ],
    )?;
  }

  conn.execute_batch("CREATE UNIQUE INDEX IF NOT EXISTS idx_files_name_unique ON files(name);")?;
  Ok(())
}
// ===== FILE: apps/desktop/src-tauri/src/db/migrations/glossary.rs =====
use rusqlite::Connection;

use super::super::columns::table_columns;

pub(crate) fn migrate_glossary_p2(conn: &Connection) -> rusqlite::Result<()> {
  conn.execute_batch(
    r#"
    CREATE TABLE IF NOT EXISTS glossary_terms (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      term TEXT NOT NULL COLLATE NOCASE UNIQUE,
      created_at_ms INTEGER NOT NULL
    );
    CREATE INDEX IF NOT EXISTS idx_glossary_terms_created ON glossary_terms(created_at_ms);
    "#,
  )?;
  Ok(())
}

pub(crate) fn migrate_glossary_gly2(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "glossary_terms")?;
  if cols.is_empty() {
    return Ok(());
  }
  if !cols.iter().any(|c| c == "note") {
    conn.execute(
      "ALTER TABLE glossary_terms ADD COLUMN note TEXT NOT NULL DEFAULT ''",
      [],
    )?;
  }
  if !cols.iter().any(|c| c == "aliases") {
    conn.execute(
      "ALTER TABLE glossary_terms ADD COLUMN aliases TEXT NOT NULL DEFAULT ''",
      [],
```

```
    )?;
  }
  if !cols.iter().any(|c| c == "domain") {
    conn.execute(
      "ALTER TABLE glossary_terms ADD COLUMN domain TEXT NOT NULL DEFAULT ''",
      [],
    )?;
  }
  if !cols.iter().any(|c| c == "updated_at_ms") {
    conn.execute(
      "ALTER TABLE glossary_terms ADD COLUMN updated_at_ms INTEGER",
      [],
    )?;
    conn.execute(
      "UPDATE glossary_terms SET updated_at_ms = created_at_ms WHERE updated_at_ms IS NULL",
      [],
    )?;
  }
  Ok(())
}

pub(crate) fn migrate_glossary_gly3(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "glossary_terms")?;
  if cols.is_empty() {
    return Ok(());
  }
  if !cols.iter().any(|c| c == "hotword_enabled") {
    conn.execute(
      "ALTER TABLE glossary_terms ADD COLUMN hotword_enabled INTEGER NOT NULL DEFAULT 1",
      [],
    )?;
  }
  conn.execute_batch(
    "CREATE INDEX IF NOT EXISTS idx_glossary_terms_hotword ON glossary_terms(hotword_enabled);",
  )?;
  Ok(())
}
// ===== FILE: apps/desktop/src-tauri/src/db/migrations/mod.rs =====
pub(crate) mod correction_memory;
pub(crate) mod edit_log;
pub(crate) mod files;
pub(crate) mod glossary;
pub(crate) mod projects;
pub(crate) mod segments;
pub(crate) mod speakers;

use rusqlite::Connection;

pub(super) fn run_incremental(conn: &Connection) -> rusqlite::Result<()> {
  segments::migrate_segments_p2(conn)?;
```

```
    segments::migrate_segments_uid(conn)?;
    segments::migrate_segments_kind(conn)?;
    segments::migrate_segments_text_stage(conn)?;
    segments::migrate_segments_annotation(conn)?;
    segments::migrate_segments_frozen(conn)?;
    speakers::migrate_speaker_diarization(conn)?;
    glossary::migrate_glossary_p2(conn)?;
    glossary::migrate_glossary_gly2(conn)?;
    glossary::migrate_glossary_gly3(conn)?;
    correction_memory::migrate_correction_memory_p2(conn)?;
    edit_log::migrate_edit_log_snapshots(conn)?;
    files::migrate_files_import_provenance(conn)?;
    files::migrate_files_duration_sec(conn)?;
    files::migrate_files_name_unique(conn)?;
    projects::migrate_projects_metadata(conn)?;
    Ok(())
}
// ===== FILE: apps/desktop/src-tauri/src/db/migrations/projects.rs =====
use rusqlite::Connection;

use super::super::columns::table_columns;

pub(super) fn migrate_projects_metadata(conn: &Connection) -> rusqlite::Result<()> {
    let cols = table_columns(conn, "projects")?;
    if cols.is_empty() {
        return Ok(());
    }
    for col in [
        "narrator",
        "recorded_at",
        "location",
        "subject",
        "transcriber",
    ] {
        if !cols.iter().any(|c| c == col) {
            conn.execute(&format!("ALTER TABLE projects ADD COLUMN {col} TEXT"), [])?;
        }
    }
    Ok(())
}
// ===== FILE: apps/desktop/src-tauri/src/db/migrations/segments.rs =====
use rusqlite::Connection;

use super::super::columns::table_columns;

pub(crate) fn migrate_segments_p2(conn: &Connection) -> rusqlite::Result<()> {
    let cols = table_columns(conn, "segments")?;
    if cols.is_empty() {
        return Ok(());
    }
```

```
    if !cols.iter().any(|c| c == "confidence") {
        conn.execute("ALTER TABLE segments ADD COLUMN confidence REAL", [])?;
    }
    if !cols.iter().any(|c| c == "low_confidence") {
        conn.execute(
            "ALTER TABLE segments ADD COLUMN low_confidence INTEGER NOT NULL DEFAULT 0",
            [],
        )?;
    }
    if !cols.iter().any(|c| c == "detail") {
        conn.execute(
            "ALTER TABLE segments ADD COLUMN detail TEXT NOT NULL DEFAULT ''",
            [],
        )?;
    }
    Ok(())
}

pub(crate) fn migrate_segments_uid(conn: &Connection) -> rusqlite::Result<()> {
    let cols = table_columns(conn, "segments")?;
    if cols.is_empty() {
        return Ok(());
    }
    if !cols.iter().any(|c| c == "uid") {
        conn.execute("ALTER TABLE segments ADD COLUMN uid TEXT", [])?;
    }
    let mut stmt = conn.prepare("SELECT id FROM segments WHERE uid IS NULL OR trim(uid) = ''")?;
    let ids = stmt
        .query_map([], |row| row.get::<_, i64>(0))?
        .collect::<Result<Vec<_>, _>>()?;
    for row_id in ids {
        let uid = uuid::Uuid::new_v4().to_string();
        conn.execute(
            "UPDATE segments SET uid = ?1 WHERE id = ?2",
            rusqlite::params![uid, row_id],
        )?;
    }
    conn.execute_batch(
        "CREATE UNIQUE INDEX IF NOT EXISTS idx_segments_file_uid ON segments(file_id, uid);",
    )?;
    Ok(())
}

pub(crate) fn migrate_segments_kind(conn: &Connection) -> rusqlite::Result<()> {
    let cols = table_columns(conn, "segments")?;
    if cols.is_empty() {
        return Ok(());
    }
    if !cols.iter().any(|c| c == "kind") {
        conn.execute("ALTER TABLE segments ADD COLUMN kind TEXT", [])?;
```

```
  }
  Ok(())
}

pub(crate) fn migrate_segments_annotation(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "segments")?;
  if cols.is_empty() {
    return Ok(());
  }
  if !cols.iter().any(|c| c == "annotation") {
    conn.execute(
      "ALTER TABLE segments ADD COLUMN annotation TEXT NOT NULL DEFAULT ''",
      [],
    )?;
  }
  Ok(())
}

pub(crate) fn migrate_segments_text_stage(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "segments")?;
  if cols.is_empty() {
    return Ok(());
  }
  if !cols.iter().any(|c| c == "text_stage") {
    conn.execute(
      "ALTER TABLE segments ADD COLUMN text_stage TEXT NOT NULL DEFAULT 'auto_transcribe'",
      [],
    )?;
  }
  if !cols.iter().any(|c| c == "finalize_via") {
    conn.execute("ALTER TABLE segments ADD COLUMN finalize_via TEXT", [])?;
  }
  Ok(())
}

pub(crate) fn migrate_segments_frozen(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "segments")?;
  if cols.is_empty() {
    return Ok(());
  }
  if !cols.iter().any(|c| c == "frozen") {
    conn.execute(
      "ALTER TABLE segments ADD COLUMN frozen INTEGER NOT NULL DEFAULT 0",
      [],
    )?;
  }
  Ok(())
}
// ===== FILE: apps/desktop/src-tauri/src/db/migrations/speakers.rs =====
use rusqlite::Connection;
```

```
use super::super::columns::table_columns;

pub(crate) fn migrate_speaker_diarization(conn: &Connection) -> rusqlite::Result<()> {
  let segment_cols = table_columns(conn, "segments")?;
  if !segment_cols.is_empty() && !segment_cols.iter().any(|c| c == "speaker_id") {
    conn.execute("ALTER TABLE segments ADD COLUMN speaker_id TEXT", [])?;
  }
  let file_cols = table_columns(conn, "files")?;
  if !file_cols.is_empty() && !file_cols.iter().any(|c| c == "diarization_run_id") {
    conn.execute("ALTER TABLE files ADD COLUMN diarization_run_id TEXT", [])?;
  }
  conn.execute_batch(
    r#"
    CREATE TABLE IF NOT EXISTS file_speakers (
      file_id TEXT NOT NULL,
      speaker_id TEXT NOT NULL,
      ordinal INTEGER NOT NULL,
      display_name TEXT,
      created_at_ms INTEGER NOT NULL,
      updated_at_ms INTEGER NOT NULL,
      PRIMARY KEY (file_id, speaker_id),
      FOREIGN KEY (file_id) REFERENCES files(id) ON DELETE CASCADE
    );
    CREATE UNIQUE INDEX IF NOT EXISTS idx_file_speakers_ordinal
      ON file_speakers(file_id, ordinal);
    "#,
  )?;
  Ok(())
}
// ===== FILE: apps/desktop/src-tauri/src/db/mod.rs =====
//! SQLite schema and migration for file-container projects (greenfield).
//! Projects are containers; files hold segments and optional audio.

mod columns;
mod migrations;
mod pool;
mod schema;

use rusqlite::Connection;

pub use pool::{
  bootstrap_db_at, configure_sqlite_connection, configure_sqlite_connection_readonly, DbPool,
};

pub fn migrate(conn: &Connection) -> rusqlite::Result<()> {
  schema::ensure_projects_schema_v2(conn)?;
  schema::ensure_segments_schema_v2(conn)?;
  schema::create_base_tables(conn)?;
  migrations::run_incremental(conn)
```

```
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::db::columns::table_columns;
    use crate::db::migrations::glossary::{
        migrate_glossary_gly2, migrate_glossary_gly3, migrate_glossary_p2,
    };
    use crate::db::migrations::segments::{migrate_segments_p2, migrate_segments_uid};
    use rusqlite::Connection;

    #[test]
    fn migrate_projects_metadata_adds_columns() {
        let conn = Connection::open_in_memory().unwrap();
        migrate(&conn).unwrap();
        let cols = table_columns(&conn, "projects").unwrap();
        assert!(cols.contains(&"narrator".to_string()));
        assert!(cols.contains(&"recorded_at".to_string()));
        assert!(cols.contains(&"location".to_string()));
        assert!(cols.contains(&"subject".to_string()));
        assert!(cols.contains(&"transcriber".to_string()));
        migrate(&conn).unwrap();
    }

    #[test]
    fn migrate_files_import_provenance_adds_columns() {
        let conn = Connection::open_in_memory().unwrap();
        migrate(&conn).unwrap();
        let cols = table_columns(&conn, "files").unwrap();
        assert!(cols.contains(&"import_source_path".to_string()));
        assert!(cols.contains(&"import_content_sha256".to_string()));
        assert!(cols.contains(&"import_source_size".to_string()));
        assert!(cols.contains(&"import_source_modified_ms".to_string()));
        migrate(&conn).unwrap();
    }

    #[test]
    fn migrate_creates_all_tables() {
        let conn = Connection::open_in_memory().unwrap();
        migrate(&conn).unwrap();

        let tables: Vec<String> = conn
            .prepare("SELECT name FROM sqlite_master WHERE type='table'")
            .unwrap()
            .query_map([], |row| row.get(0))
            .unwrap()
            .collect::<Result<Vec<_>, _>>()
            .unwrap();
```

```
    assert!(tables.contains(&"projects".to_string()));
    assert!(tables.contains(&"files".to_string()));
    assert!(tables.contains(&"segments".to_string()));
    assert!(tables.contains(&"edit_log".to_string()));
    assert!(tables.contains(&"edit_log_snapshots".to_string()));
    assert!(tables.contains(&"glossary_terms".to_string()));
    assert!(tables.contains(&"correction_memory".to_string()));
}

#[test]
fn migrate_is_idempotent() {
    let conn = Connection::open_in_memory().unwrap();
    migrate(&conn).unwrap();
    migrate(&conn).unwrap();
}

#[test]
fn migrate_glossary_gly2_adds_metadata_columns() {
    let conn = Connection::open_in_memory().unwrap();
    migrate_glossary_p2(&conn).unwrap();
    let cols_before = table_columns(&conn, "glossary_terms").unwrap();
    assert!(!cols_before.iter().any(|c| c == "note"));

    migrate_glossary_gly2(&conn).unwrap();
    migrate_glossary_gly2(&conn).unwrap();

    let cols = table_columns(&conn, "glossary_terms").unwrap();
    assert!(cols.contains(&"note".to_string()));
    assert!(cols.contains(&"aliases".to_string()));
    assert!(cols.contains(&"domain".to_string()));
    assert!(cols.contains(&"updated_at_ms".to_string()));
}

#[test]
fn migrate_glossary_gly3_adds_hotword_enabled() {
    let conn = Connection::open_in_memory().unwrap();
    migrate_glossary_p2(&conn).unwrap();
    migrate_glossary_gly2(&conn).unwrap();
    migrate_glossary_gly3(&conn).unwrap();
    migrate_glossary_gly3(&conn).unwrap();
    let cols = table_columns(&conn, "glossary_terms").unwrap();
    assert!(cols.contains(&"hotword_enabled".to_string()));
}

#[test]
fn migrate_segments_p2_adds_columns_to_existing_table() {
    let conn = Connection::open_in_memory().unwrap();
    conn.execute_batch(
        r#"
        CREATE TABLE segments (
```

```
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            file_id TEXT NOT NULL,
            idx INTEGER NOT NULL,
            start_sec REAL NOT NULL DEFAULT 0,
            end_sec REAL NOT NULL DEFAULT 0,
            text TEXT NOT NULL DEFAULT ''
        );
        "#,
    )
    .unwrap();

    migrate_segments_p2(&conn).unwrap();

    let cols = table_columns(&conn, "segments").unwrap();
    assert!(cols.contains(&"confidence".to_string()));
    assert!(cols.contains(&"low_confidence".to_string()));
    assert!(cols.contains(&"detail".to_string()));
}

#[test]
fn migrate_drops_old_segments_table_without_file_id() {
    let conn = Connection::open_in_memory().unwrap();
    conn.execute_batch(
        r#"
        CREATE TABLE projects (
            id TEXT PRIMARY KEY NOT NULL,
            name TEXT NOT NULL,
            created_at_ms INTEGER NOT NULL,
            updated_at_ms INTEGER NOT NULL
        );
        CREATE TABLE segments (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            project_id TEXT NOT NULL,
            idx INTEGER NOT NULL,
            start_sec REAL NOT NULL DEFAULT 0,
            end_sec REAL NOT NULL DEFAULT 0,
            text TEXT NOT NULL DEFAULT ''
        );
        "#,
    )
    .unwrap();

    migrate(&conn).unwrap();

    let cols = table_columns(&conn, "segments").unwrap();
    assert!(cols.contains(&"file_id".to_string()));
    assert!(!cols.contains(&"project_id".to_string()));
}

#[test]
```

```
fn migrate_preserves_segments_with_file_id() {
    let conn = Connection::open_in_memory().unwrap();
    conn.execute_batch(
        r#"
        CREATE TABLE files (
            id TEXT PRIMARY KEY NOT NULL,
            project_id TEXT NOT NULL,
            name TEXT NOT NULL,
            file_type TEXT NOT NULL,
            created_at_ms INTEGER NOT NULL,
            updated_at_ms INTEGER NOT NULL
        );
        CREATE TABLE segments (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            file_id TEXT NOT NULL,
            idx INTEGER NOT NULL,
            start_sec REAL NOT NULL DEFAULT 0,
            end_sec REAL NOT NULL DEFAULT 0,
            text TEXT NOT NULL DEFAULT ''
        );
        "#,
    )
    .unwrap();
    conn.execute(
        "INSERT INTO files (id, project_id, name, file_type, created_at_ms, updated_at_ms) VA...
        ["f1", "p1", "test", "text", "0", "0"],
    )
    .unwrap();
    conn.execute(
        "INSERT INTO segments (file_id, idx, start_sec, end_sec, text) VALUES (?1, ?2, ?3, ?4...
        ["f1", "0", "0.0", "1.0", "hello"],
    )
    .unwrap();

    migrate(&conn).unwrap();

    let count: i64 = conn
        .query_row("SELECT COUNT(*) FROM segments", [], |r| r.get(0))
        .unwrap();
    assert_eq!(count, 1);
}

#[test]
fn migrate_drops_old_projects_table_with_audio_storage_path() {
    let conn = Connection::open_in_memory().unwrap();
    conn.execute_batch(
        r#"
        CREATE TABLE projects (
            id TEXT PRIMARY KEY NOT NULL,
            name TEXT NOT NULL,
```

```
            audio_storage_path TEXT NOT NULL,
            created_at_ms INTEGER NOT NULL,
            updated_at_ms INTEGER NOT NULL
        );
        CREATE TABLE segments (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            project_id TEXT NOT NULL,
            idx INTEGER NOT NULL,
            start_sec REAL NOT NULL DEFAULT 0,
            end_sec REAL NOT NULL DEFAULT 0,
            text TEXT NOT NULL DEFAULT ''
        );
        "#,
    )
    .unwrap();
    conn.execute(
        "INSERT INTO projects (id, name, audio_storage_path, created_at_ms, updated_at_ms) VA...
        ["p1", "Old", "/tmp/audio.wav", "0", "0"],
    )
    .unwrap();

    migrate(&conn).unwrap();

    let cols = table_columns(&conn, "projects").unwrap();
    assert!(!cols.contains(&"audio_storage_path".to_string()));
    assert!(cols.contains(&"name".to_string()));

    let count: i64 = conn
        .query_row("SELECT COUNT(*) FROM projects", [], |r| r.get(0))
        .unwrap();
    assert_eq!(count, 0);
}

#[test]
fn migrate_files_name_unique_creates_index_when_no_duplicates() {
    let conn = Connection::open_in_memory().unwrap();
    migrate(&conn).unwrap();
    conn.execute(
        "INSERT INTO projects (id, name, created_at_ms, updated_at_ms) VALUES ('p', 'P', 0, 0)",
        [],
    )
    .unwrap();
    conn.execute(
        "INSERT INTO files (id, project_id, name, file_type, created_at_ms, updated_at_ms) \
         VALUES ('f1', 'p', 'unique-name', 'text', 0, 0)",
        [],
    )
    .unwrap();

    let index_count: i64 = conn
```

```
        .query_row(
            "SELECT COUNT(*) FROM sqlite_master WHERE type = 'index' AND name = 'idx_files_na...
            [],
            |r| r.get(0),
        )
        .unwrap();
    assert_eq!(index_count, 1);

    let result = conn.execute(
        "INSERT INTO files (id, project_id, name, file_type, created_at_ms, updated_at_ms) \
         VALUES ('f2', 'p', 'unique-name', 'text', 1, 1)",
        [],
    );
    assert!(
        result.is_err(),
        "duplicate name insert should be rejected by unique index"
    );
}

#[test]
fn migrate_files_name_unique_dedupes_pre_existing_duplicates_and_logs_edit_log() {
    use crate::db::migrations::files::migrate_files_name_unique;

    let conn = Connection::open_in_memory().unwrap();
    conn.execute_batch(
        r#"
        PRAGMA foreign_keys = ON;
        CREATE TABLE projects (
            id TEXT PRIMARY KEY NOT NULL,
            name TEXT NOT NULL,
            created_at_ms INTEGER NOT NULL,
            updated_at_ms INTEGER NOT NULL
        );
        CREATE TABLE files (
            id TEXT PRIMARY KEY NOT NULL,
            project_id TEXT NOT NULL,
            name TEXT NOT NULL,
            file_type TEXT NOT NULL,
            created_at_ms INTEGER NOT NULL,
            updated_at_ms INTEGER NOT NULL
        );
        CREATE TABLE edit_log (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            project_id TEXT NOT NULL,
            at_ms INTEGER NOT NULL,
            kind TEXT NOT NULL,
            detail TEXT NOT NULL
        );
        "#,
    )
```

```
    .unwrap();
    conn.execute(
        "INSERT INTO projects (id, name, created_at_ms, updated_at_ms) VALUES ('p', 'P', 0, 0)",
        [],
    )
    .unwrap();
    // Two pre-existing rows already share the same name (historical race / bug).
    conn.execute(
        "INSERT INTO files (id, project_id, name, file_type, created_at_ms, updated_at_ms) \
         VALUES ('f1', 'p', 'dup', 'text', 100, 100)",
        [],
    )
    .unwrap();
    conn.execute(
        "INSERT INTO files (id, project_id, name, file_type, created_at_ms, updated_at_ms) \
         VALUES ('f2', 'p', 'dup', 'text', 200, 200)",
        [],
    )
    .unwrap();

    migrate_files_name_unique(&conn).unwrap();

    let f1_name: String = conn
        .query_row("SELECT name FROM files WHERE id = 'f1'", [], |r| r.get(0))
        .unwrap();
    let f2_name: String = conn
        .query_row("SELECT name FROM files WHERE id = 'f2'", [], |r| r.get(0))
        .unwrap();
    assert_eq!(
        f1_name, "dup",
        "earliest-created row keeps its original name"
    );
    assert_eq!(
        f2_name, "dup (2)",
        "later row is renamed to a Finder-style suffix"
    );

    let log_count: i64 = conn
        .query_row(
            "SELECT COUNT(*) FROM edit_log WHERE kind = 'migration_dedupe_file_name'",
            [],
            |r| r.get(0),
        )
        .unwrap();
    assert_eq!(log_count, 1);

    let index_count: i64 = conn
        .query_row(
            "SELECT COUNT(*) FROM sqlite_master WHERE type = 'index' AND name = 'idx_files_na...
            [],
```

```
            |r| r.get(0),
        )
        .unwrap();
    assert_eq!(index_count, 1);

    // Idempotent: running again with the index already present is a no-op.
    migrate_files_name_unique(&conn).unwrap();
}

#[test]
fn migrate_segments_uid_backfills_two_rows_same_file_before_unique_index() {
    let conn = Connection::open_in_memory().unwrap();
    conn.execute_batch(
        r#"
        CREATE TABLE files (
            id TEXT PRIMARY KEY NOT NULL,
            project_id TEXT NOT NULL,
            name TEXT NOT NULL,
            file_type TEXT NOT NULL,
            created_at_ms INTEGER NOT NULL,
            updated_at_ms INTEGER NOT NULL
        );
        CREATE TABLE segments (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            file_id TEXT NOT NULL,
            idx INTEGER NOT NULL,
            start_sec REAL NOT NULL DEFAULT 0,
            end_sec REAL NOT NULL DEFAULT 0,
            text TEXT NOT NULL DEFAULT ''
        );
        "#,
    )
    .unwrap();
    conn.execute(
        "INSERT INTO files (id, project_id, name, file_type, created_at_ms, updated_at_ms) VA...
        ["f1", "p1", "test", "text", "0", "0"],
    )
    .unwrap();
    conn.execute(
        "INSERT INTO segments (file_id, idx, start_sec, end_sec, text) VALUES (?1, ?2, ?3, ?4...
        ["f1", "0", "0.0", "1.0", "a"],
    )
    .unwrap();
    conn.execute(
        "INSERT INTO segments (file_id, idx, start_sec, end_sec, text) VALUES (?1, ?2, ?3, ?4...
        ["f1", "1", "1.0", "2.0", "b"],
    )
    .unwrap();

    migrate_segments_uid(&conn).unwrap();
```

```
    let uids: Vec<String> = conn
      .prepare("SELECT uid FROM segments WHERE file_id = 'f1' ORDER BY idx")
      .unwrap()
      .query_map([], |row| row.get(0))
      .unwrap()
      .collect::<Result<Vec<_>, _>>()
      .unwrap();
    assert_eq!(uids.len(), 2);
    assert!(!uids[0].is_empty());
    assert!(!uids[1].is_empty());
    assert_ne!(uids[0], uids[1]);

    let index_count: i64 = conn
      .query_row(
        "SELECT COUNT(*) FROM sqlite_master WHERE type = 'index' AND name = 'idx_segments...
        [],
        |r| r.get(0),
      )
      .unwrap();
    assert_eq!(index_count, 1);
  }
}
// ===== FILE: apps/desktop/src-tauri/src/db/pool.rs =====
use std::path::Path;
use std::time::Duration;

use r2d2::Pool;
use r2d2_sqlite::SqliteConnectionManager;
use rusqlite::Connection;

use super::migrate;

pub type DbPool = Pool<SqliteConnectionManager>;

/// Shared PRAGMA for any non-pooled SQLite connection (WAL + busy_timeout + FK).
pub fn configure_sqlite_connection(conn: &Connection) -> rusqlite::Result<()> {
  conn.pragma_update(None, "foreign_keys", "ON")?;
  conn.pragma_update(None, "journal_mode", "WAL")?;
  conn.busy_timeout(Duration::from_millis(5000))?;
  Ok(())
}

/// Read-only opens: busy_timeout only (no journal_mode change on RO handles).
pub fn configure_sqlite_connection_readonly(conn: &Connection) -> rusqlite::Result<()> {
  conn.busy_timeout(Duration::from_millis(5000))?;
  Ok(())
}

fn configure_pooled_connection(conn: &mut Connection) -> rusqlite::Result<()> {
```

```
    configure_sqlite_connection(conn)
}

pub fn open_pool(db_path: &Path) -> Result<DbPool, String> {
    let manager = SqliteConnectionManager::file(db_path).with_init(configure_pooled_connection);
    Pool::builder()
        .max_size(8)
        .build(manager)
        .map_err(|e| e.to_string())
}

/// Migrate (idempotent), then return a WAL-enabled connection pool.
pub fn bootstrap_db_at(db_path: &Path) -> Result<DbPool, String> {
    if let Some(parent) = db_path.parent() {
        std::fs::create_dir_all(parent).map_err(|e| e.to_string())?;
    }
    {
        let conn = Connection::open(db_path).map_err(|e| e.to_string())?;
        migrate(&conn).map_err(|e| e.to_string())?;
    }
    let pool = open_pool(db_path)?;
    // Defer SHA256 provenance backfill so startup migrate is not blocked hashing every audio file.
    spawn_import_provenance_backfill(pool.clone());
    Ok(pool)
}

fn spawn_import_provenance_backfill(pool: DbPool) {
    std::thread::spawn(move || {
        let conn = match pool.get() {
            Ok(c) => c,
            Err(e) => {
                eprintln!("[db] backfill_files_import_provenance: pool get failed: {e}");
                return;
            }
        };
        if let Err(e) = crate::project::import_duplicate::backfill_files_import_provenance(&conn) {
            eprintln!("[db] backfill_files_import_provenance: {e}");
        }
    });
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::fs;
    use uuid::Uuid;

    #[test]
    fn bootstrap_db_at_enables_wal_on_pooled_connections() {
        let root = std::env::temp_dir().join(format!("rushi-db-pool-{}", Uuid::new_v4()));
```

```
    fs::create_dir_all(&root).unwrap();
    let db_path = root.join("rushi.sqlite3");
    let pool = bootstrap_db_at(&db_path).expect("bootstrap pool");
    let conn = pool.get().expect("pool conn");
    let mode: String = conn
      .query_row("PRAGMA journal_mode", [], |row| row.get(0))
      .expect("journal_mode");
    assert_eq!(mode.to_ascii_lowercase(), "wal");
    let _ = fs::remove_dir_all(&root);
  }
}
// ===== FILE: apps/desktop/src-tauri/src/db/schema.rs =====
use rusqlite::Connection;

use super::columns::table_columns;

/// Greenfield: if an old project-centric projects table (with audio_storage_path) exists,
/// drop all dependent tables so the new file-centric schema can be created.
pub(super) fn ensure_projects_schema_v2(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "projects")?;
  if !cols.is_empty() && cols.iter().any(|c| c == "audio_storage_path") {
    conn.execute("PRAGMA foreign_keys = OFF", [])?;
    conn.execute("DROP TABLE IF EXISTS segments", [])?;
    conn.execute("DROP TABLE IF EXISTS files", [])?;
    conn.execute("DROP TABLE IF EXISTS edit_log", [])?;
    conn.execute("DROP TABLE IF EXISTS projects", [])?;
    conn.execute("PRAGMA foreign_keys = ON", [])?;
  }
  Ok(())
}

/// Greenfield: if an old project-centric segments table (without file_id) exists,
/// drop it so the new file-centric schema can be created.
pub(super) fn ensure_segments_schema_v2(conn: &Connection) -> rusqlite::Result<()> {
  let cols = table_columns(conn, "segments")?;
  if !cols.is_empty() && !cols.iter().any(|c| c == "file_id") {
    conn.execute("DROP INDEX IF EXISTS idx_segments_file", [])?;
    conn.execute("DROP TABLE IF EXISTS segments", [])?;
  }
  Ok(())
}

pub(super) fn create_base_tables(conn: &Connection) -> rusqlite::Result<()> {
  conn.execute_batch(
    r#"
    PRAGMA foreign_keys = ON;
    PRAGMA busy_timeout = 5000;

    CREATE TABLE IF NOT EXISTS projects (
      id TEXT PRIMARY KEY NOT NULL,
```

```
    name TEXT NOT NULL,
    created_at_ms INTEGER NOT NULL,
    updated_at_ms INTEGER NOT NULL
  );

  CREATE TABLE IF NOT EXISTS files (
    id TEXT PRIMARY KEY NOT NULL,
    project_id TEXT NOT NULL,
    name TEXT NOT NULL,
    file_type TEXT NOT NULL CHECK(file_type IN ('text', 'paired', 'audio_only')),
    audio_path TEXT,
    diarization_run_id TEXT,
    created_at_ms INTEGER NOT NULL,
    updated_at_ms INTEGER NOT NULL,
    FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE
  );
  CREATE INDEX IF NOT EXISTS idx_files_project ON files(project_id);

  CREATE TABLE IF NOT EXISTS segments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    file_id TEXT NOT NULL,
    idx INTEGER NOT NULL,
    start_sec REAL NOT NULL DEFAULT 0,
    end_sec REAL NOT NULL DEFAULT 0,
    text TEXT NOT NULL DEFAULT '',
    confidence REAL,
    low_confidence INTEGER NOT NULL DEFAULT 0,
    detail TEXT NOT NULL DEFAULT '',
    uid TEXT NOT NULL DEFAULT '',
    kind TEXT,
    speaker_id TEXT,
    FOREIGN KEY (file_id) REFERENCES files(id) ON DELETE CASCADE,
    UNIQUE (file_id, idx)
  );
  CREATE INDEX IF NOT EXISTS idx_segments_file ON segments(file_id);

  CREATE TABLE IF NOT EXISTS file_speakers (
    file_id TEXT NOT NULL,
    speaker_id TEXT NOT NULL,
    ordinal INTEGER NOT NULL,
    display_name TEXT,
    created_at_ms INTEGER NOT NULL,
    updated_at_ms INTEGER NOT NULL,
    PRIMARY KEY (file_id, speaker_id),
    FOREIGN KEY (file_id) REFERENCES files(id) ON DELETE CASCADE
  );
  CREATE UNIQUE INDEX IF NOT EXISTS idx_file_speakers_ordinal
    ON file_speakers(file_id, ordinal);

  CREATE TABLE IF NOT EXISTS edit_log (
```

```
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        project_id TEXT NOT NULL,
        at_ms INTEGER NOT NULL,
        kind TEXT NOT NULL,
        detail TEXT NOT NULL,
        FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE
      );
      "#,
  )?;
  Ok(())
}
// ===== FILE: apps/desktop/src-tauri/src/native_audio/clock.rs =====
use std::sync::atomic::{AtomicBool, AtomicU32, AtomicU64, Ordering};

/// Shared playhead / transport flags between engine, decode, and CPAL callback.
pub(crate) struct SharedClock {
  pub playing: AtomicBool,
  pub play_requested: AtomicBool,
  pub buffer_ready: AtomicBool,
  pub stop: AtomicBool,
  pub seek_seq: AtomicU64,
  pub drain_seq: AtomicU64,
  pub rate_seq: AtomicU64,
  /// Output is flushing stale ring PCM after seek/rebuild; decode must not push.
  pub drain_pending: AtomicBool,
  pub position_us: AtomicU64,
  pub duration_us: AtomicU64,
  pub rate_milli: AtomicU32,
  pub output_sample_rate: AtomicU32,
  pub output_channels: AtomicU32,
  /// Demuxer hit EOF; next Play must re-seek before decoding again.
  pub at_eof: AtomicBool,
  /// Set when decode/output reports a soft underrun (telemetry / Error event).
  pub underrun: AtomicBool,
  pub underrun_reported: AtomicBool,
  pub queued_samples: AtomicU64,
}

impl SharedClock {
  pub(crate) fn new(duration_sec: f64, sample_rate: u32, channels: u32) -> Self {
    Self {
      playing: AtomicBool::new(false),
      play_requested: AtomicBool::new(false),
      buffer_ready: AtomicBool::new(false),
      stop: AtomicBool::new(false),
      seek_seq: AtomicU64::new(0),
      drain_seq: AtomicU64::new(0),
      rate_seq: AtomicU64::new(0),
      drain_pending: AtomicBool::new(false),
      position_us: AtomicU64::new(0),
```

```
            duration_us: AtomicU64::new((duration_sec.max(0.0) * 1_000_000.0) as u64),
            rate_milli: AtomicU32::new(1000),
            output_sample_rate: AtomicU32::new(sample_rate.max(1)),
            output_channels: AtomicU32::new(channels.max(1)),
            at_eof: AtomicBool::new(false),
            underrun: AtomicBool::new(false),
            underrun_reported: AtomicBool::new(false),
            queued_samples: AtomicU64::new(0),
        }
    }

    pub(crate) fn current_time_sec(&self) -> f64 {
        self.position_us.load(Ordering::Relaxed) as f64 / 1_000_000.0
    }

    pub(crate) fn duration_sec(&self) -> f64 {
        self.duration_us.load(Ordering::Relaxed) as f64 / 1_000_000.0
    }

    pub(crate) fn rate(&self) -> f32 {
        self.rate_milli.load(Ordering::Relaxed) as f32 / 1000.0
    }

    pub(crate) fn decrement_queued_samples(&self, n: u64) {
        let _ = self
            .queued_samples
            .try_update(Ordering::Relaxed, Ordering::Relaxed, |cur| {
                Some(cur.saturating_sub(n))
            });
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn shared_clock_rate_roundtrip() {
        let c = SharedClock::new(10.0, 48000, 2);
        c.rate_milli.store(1500, Ordering::SeqCst);
        assert!((c.rate() - 1.5).abs() < 0.001);
    }
}
// ===== FILE: apps/desktop/src-tauri/src/native_audio/commands.rs =====
use tauri::ipc::Channel;
use tauri::State;

use crate::project::audio_container_normalize::normalize_project_audio_in_place;
use crate::project::utils::{append_desktop_log_line, canonicalize_audio_storage_path, open_db};
use crate::DbState;
```

```
// ===== ... middle omitted (154314 lines) for 软著鉴别材料 前30页+后30页 ... =====
    lane: args.laneByIndex[draft.idx] ?? 0,
    laneCount: args.laneCount,
    containerHeightPx: args.layoutHeightPx,
  });
  setCspLayoutRules(previewEl, {
    display: "block",
    left: geom.leftPx,
    width: geom.widthPx,
    top: geom.topPx,
    height: geom.heightPx,
    bottom: null,
    background: waveformRegionFillColor(seg, true),
    border: "none",
    borderLeft: "none",
    borderRight: "none",
    zIndex: 4,
  });
}
// ===== FILE: apps/desktop/src/utils/waveformSegmentOverlayGeometry.ts =====
import type { SegmentDto } from "../tauri/projectApi";
import { computeDragSegmentBounds } from "./waveformSegmentBounds";
import type { OverlayDragMode } from "./waveformSegmentOverlayGestures";
import { timeToTimelinePx } from "./waveformProjection";

export type SegmentOverlayDraft = {
  idx: number;
  startSec: number;
  endSec: number;
  neighborPatches?: Array<{ idx: number; startSec: number; endSec: number }>;
  pendingDeleteIndices?: number[];
};

export type CreateRangePreview = {
  startSec: number;
  endSec: number;
};

export type OverlayDragState = {
  mode: OverlayDragMode;
  pointerId: number;
  segmentIdx: number;
  anchorTimeSec: number;
  anchorClientX: number;
  initialStartSec: number;
  initialEndSec: number;
  moved: boolean;
  /** React SC1 at pointerdown — tap select vs seek-within must not use post-preview SC1. */
  selectedIdxAtPointerDown: number;
  /** pointerdown gesture down 已 seek+reveal（pointerup 不得 seek-within）。 */
```

```
  viewportSyncedOnDown?: boolean;
  /** Stable pointerdown→pointerup session id for preview dedupe. */
  sessionId?: string;
  /** Blank overlay shell lasso — user intent is create unless Shift extends selection. */
  blankLasso?: boolean;
  /** pointerdown already ran seekBlankToTime — pointerup short tap must not re-seek. */
  blankSeekedOnDown?: boolean;
  baseIndices?: Set<number>;
  lastFinalizedBounds?: { startSec: number; endSec: number };
};

  /** 语段条渲染边界：拖拽 draft 优先于 committed segment。 */
export function resolveSegmentBoundsAt(
  idx: number,
  segments: SegmentDto[],
  segmentDraft: SegmentOverlayDraft | null,
): { startSec: number; endSec: number } | null {
  if (segmentDraft?.pendingDeleteIndices?.includes(idx)) {
    return null;
  }
  const seg = segments[idx];
  if (!seg) return null;
  if (segmentDraft?.idx === idx) {
    return { startSec: segmentDraft.startSec, endSec: segmentDraft.endSec };
  }
  const patch = segmentDraft?.neighborPatches?.find((p) => p.idx === idx);
  if (patch) {
    return { startSec: patch.startSec, endSec: patch.endSec };
  }
  return { startSec: seg.start_sec, endSec: seg.end_sec };
}

/** 框选预览 DOM 几何（timeline 坐标，随 tier 滚动）。 */
export function computeCreatePreviewStyle(input: {
  createPreview: CreateRangePreview;
  timelineWidthPx: number;
  durationSec: number;
}): { left: number; width: number } {
  const lo = Math.min(input.createPreview.startSec, input.createPreview.endSec);
  const hi = Math.max(input.createPreview.startSec, input.createPreview.endSec);
  const leftPx = timeToTimelinePx(lo, input.timelineWidthPx, input.durationSec);
  const rightPx = timeToTimelinePx(hi, input.timelineWidthPx, input.durationSec);
  return {
    left: Math.max(0, leftPx),
    width: Math.max(2, rightPx - leftPx),
  };
}

export function boundsForOverlayDrag(
  drag: OverlayDragState,
```

```
  timeSec: number,
  durationSec: number,
): { startSec: number; endSec: number } | null {
  if (drag.mode === "lasso") return null;
  const delta = timeSec - drag.anchorTimeSec;
  return computeDragSegmentBounds(
    drag.mode,
    drag.initialStartSec,
    drag.initialEndSec,
    delta,
    durationSec || drag.initialEndSec,
  );
}
// ===== FILE: apps/desktop/src/utils/waveformSegmentOverlayGestures.ts =====
import { WAVEFORM_SEGMENT_MIN_SPAN_SEC, type SegmentDragMode } from "./waveformSegmentBounds";
import type { SegmentOverlapPolicy } from "./segmentTimeRange";

export type OverlayDragMode = SegmentDragMode | "lasso";

/** Blank overlay shell pointer-down: lasso select/create (解语 wave lasso) or seek. */
export type BlankOverlayShellDragMode = "lasso" | "seek";

export function resolveBlankOverlayShellDragMode(input: {
  enableCreateRange: boolean;
  hasOnCreateRange: boolean;
}): BlankOverlayShellDragMode {
  if (input.enableCreateRange && input.hasOnCreateRange) return "lasso";
  return "seek";
}

export type OverlayPointerUpIntent =
  | { kind: "select-segment"; segmentIdx: number; pointerTimeSec: number }
  | {
      kind: "commit-bounds";
      segmentIdx: number;
      startSec: number;
      endSec: number;
      neighborPatches?: Array<{ idx: number; startSec: number; endSec: number }>;
      deleteIndices?: number[];
    }
  | { kind: "create-range"; startSec: number; endSec: number; overlapPolicy?: SegmentOverlapPolicy }
  | { kind: "seek-blank"; timeSec: number }
  | { kind: "noop" };

/** pointerup 意图（纯函数，供 overlay 与单测共用）。 */
export function resolveOverlayPointerUpIntent(input: {
  mode: OverlayDragMode;
  moved: boolean;
  segmentIdx: number;
  pointerTimeSec: number;
```

```
  anchorTimeSec: number;
  initialStartSec: number;
  initialEndSec: number;
  clampedStartSec: number;
  clampedEndSec: number;
  minSpanSec?: number;
  neighborPatches?: Array<{ idx: number; startSec: number; endSec: number }>;
  deleteIndices?: number[];
}): OverlayPointerUpIntent {
  const minSpan = input.minSpanSec ?? WAVEFORM_SEGMENT_MIN_SPAN_SEC;

  if (input.mode === "lasso") {
    if (!input.moved) {
      return { kind: "seek-blank", timeSec: input.pointerTimeSec };
    }
    const lo = Math.min(input.initialStartSec, input.pointerTimeSec);
    const hi = Math.max(input.initialStartSec, input.pointerTimeSec);
    if (Math.abs(hi - lo) >= minSpan) {
      return { kind: "create-range", startSec: input.clampedStartSec, endSec: input.clampedEndSec };
    }
    return { kind: "seek-blank", timeSec: input.pointerTimeSec };
  }

  if (!input.moved) {
    return {
      kind: "select-segment",
      segmentIdx: input.segmentIdx,
      pointerTimeSec: input.anchorTimeSec,
    };
  }

  const unchanged =
    Math.abs(input.clampedStartSec - input.initialStartSec) < 0.0005 &&
    Math.abs(input.clampedEndSec - input.initialEndSec) < 0.0005 &&
    (input.neighborPatches?.length ?? 0) === 0 &&
    (input.deleteIndices?.length ?? 0) === 0;
  if (unchanged) {
    return {
      kind: "select-segment",
      segmentIdx: input.segmentIdx,
      pointerTimeSec: input.anchorTimeSec,
    };
  }

  return {
    kind: "commit-bounds",
    segmentIdx: input.segmentIdx,
    startSec: input.clampedStartSec,
    endSec: input.clampedEndSec,
    neighborPatches: input.neighborPatches,
```

```
  deleteIndices: input.deleteIndices,
 };
}
// ===== FILE: apps/desktop/src/utils/waveformSegmentOverlayVisibility.ts =====
import type { SegmentDto } from "../tauri/projectApi";
import {
 MAX_DOM_OVERLAY_SPARSE,
 resolveWaveformSelectionRenderProjection,
 selectWaveformOverlayInteractiveIndices,
} from "../services/waveform/waveformSelectionRenderProjection";

/** Max DOM overlay regions for sparse multi-select; larger sets rely on canvas band. */
export { MAX_DOM_OVERLAY_SPARSE };

export function selectOverlayRenderedSegmentIndices(input: {
 segments: SegmentDto[];
 dominantSpanIndices?: readonly number[];
}): number[] {
 const dominant = new Set(input.dominantSpanIndices ?? []);
 const out: number[] = [];
 for (let idx = 0; idx < input.segments.length; idx += 1) {
  if (!dominant.has(idx)) out.push(idx);
 }
 return out;
}

/** DOM overlay / canvas skip indices: explicit set + contiguous range + draft. */
export function selectOverlayInteractiveSegmentIndices(input: {
 segmentCount: number;
 selectedIdx: number;
 selectedIndices?: ReadonlySet<number>;
 selectionLo?: number;
 selectionHi?: number;
 selectionCount?: number;
 isContiguousSelection?: boolean;
 draftIdx: number | null;
}): number[] {
 return selectWaveformOverlayInteractiveIndices(input);
}

/**
 * Band canvas skip set — DOM overlay indices; primary skipped when overlay node exists.
 * When overlay DOM is missing, primary stays band-painted to avoid a white gap before React mount.
 */
export function resolveSegmentBandCanvasSkipIndexSet(input: {
 segmentCount: number;
 selectedIdx: number;
 selectedIndices?: ReadonlySet<number>;
 selectionLo?: number;
 selectionHi?: number;
```

```
 selectionCount?: number;
 isContiguousSelection?: boolean;
 draftIdx: number | null;
 overlayRoot?: ParentNode | null;
}): ReadonlySet<number> {
 return resolveWaveformSelectionRenderProjection(input).canvasSkipIndexSet;
}
// ===== FILE: apps/desktop/src/utils/waveformSegmentPackable.ts =====
import type { SegmentDto, SegmentKind } from "../tauri/projectApi";
import { roundSec3 } from "./boundsSignature";

export const WAVEFORM_SEGMENT_MIN_SPAN_SEC = 0.05;

/** Span ratio above which a segment is treated as whole-track placeholder for waveform UI. */
export const WAVEFORM_DOMINANT_SPAN_RATIO = 0.85;

function isDominantWaveformSpanSegment(
 startSec: number,
 endSec: number,
 durationSec: number,
): boolean {
 if (!(durationSec > 0)) return false;
 const lo = Math.min(startSec, endSec);
 const hi = Math.max(startSec, endSec);
 const span = hi - lo;
 if (span <= 0) return false;
 return span / durationSec >= WAVEFORM_DOMINANT_SPAN_RATIO;
}

type PlaceholderProbe = Pick<SegmentDto, "start_sec" | "end_sec"> & {
 kind?: SegmentKind | null;
};

/**
 * 是否为整轨占位语段（波形上不渲染）。**显式 `kind` 优先**：`placeholder` 即占位、
 * `speech` 即非占位（即便跨度很大也不隐藏，消除短片段长单段的假阳性）；缺省时回退
 * 0.85 跨度启发式（兼容旧数据 / 未标记语段）。
 */
export function isPlaceholderSegment(seg: PlaceholderProbe, durationSec: number): boolean {
 const kind = seg.kind === "placeholder" || seg.kind === "speech" ? seg.kind : undefined;
 if (kind === "placeholder") return true;
 if (kind === "speech") return false;
 return isDominantWaveformSpanSegment(seg.start_sec, seg.end_sec, durationSec);
}

export type PackableSegmentPartition = {
 /** Source indices kept for waveform UI (render / lane packing / hit-test / create-overlap). */
 packableIndices: number[];
 /** Source indices treated as whole-track placeholders, hidden from the waveform UI. */
 dominantSpanIndices: number[];
```

```
};

/**
 * Single authority for "which segments participate in the waveform UI" (TRUTH).
 *
 * Whole-track placeholder spans (e.g. the pre-segmentation ASR span) are excluded so
 * that rendering, lane packing, pointer hit-test and create-range overlap all agree on
 * the exact same working set. Routing every consumer through this selector prevents the
 * class of bug where the overlay hides a placeholder yet editing logic still counts it.
 *
 * Pass durationSec <= 0 (unknown duration) to keep every segment packable.
 */
export function selectPackableSegmentIndices(
  segments: ReadonlyArray<PlaceholderProbe>,
  durationSec: number,
): PackableSegmentPartition {
  const packableIndices: number[] = [];
  const dominantSpanIndices: number[] = [];
  for (let i = 0; i < segments.length; i += 1) {
    const seg = segments[i];
    if (!seg) continue;
    if (isPlaceholderSegment(seg, durationSec)) {
      dominantSpanIndices.push(i);
    } else {
      packableIndices.push(i);
    }
  }
  return { packableIndices, dominantSpanIndices };
}

/** Packable segments (identity-preserving) derived from {@link selectPackableSegmentIndices}. */
export function selectPackableSegments<T extends PlaceholderProbe>(
  segments: ReadonlyArray<T>,
  durationSec: number,
): T[] {
  const { packableIndices } = selectPackableSegmentIndices(segments, durationSec);
  return packableIndices.map((i) => segments[i]);
}

/** Packable segment time spans for long-media median default zoom. */
export function collectPackableSegmentSpansSec(
  segments: ReadonlyArray<PlaceholderProbe>,
  durationSec: number,
): number[] {
  const packable = selectPackableSegments(segments, durationSec);
  const spans: number[] = [];
  for (const seg of packable) {
    const span = Math.abs(seg.end_sec - seg.start_sec);
    if (Number.isFinite(span) && span > 0) spans.push(span);
  }
```

```
  return spans;
}

export function clampSegmentTimeBounds(
  startSec: number,
  endSec: number,
  durationSec: number,
): { startSec: number; endSec: number } {
  const lo = Math.min(startSec, endSec);
  const hi = Math.max(startSec, endSec);
  const dur = Math.max(durationSec, WAVEFORM_SEGMENT_MIN_SPAN_SEC);
  const clampedStart = roundSec3(Math.max(0, lo));
  const clampedEnd = roundSec3(Math.min(Math.max(clampedStart + WAVEFORM_SEGMENT_MIN_SPAN_SEC, hi...
  return { startSec: clampedStart, endSec: clampedEnd };
}
// ===== FILE: apps/desktop/src/utils/waveformShortcutContext.ts =====
import { TRANSCRIPT_TEXTAREA_SELECTOR } from "../pages/flushSegmentTextDrafts";
import { isTranscriptEditorCoreTarget } from "../components/editor/core/transcriptEditorDom";

function isTranscriptTextEditNode(node: Node | null): boolean {
  if (!(node instanceof HTMLElement)) return false;
  if (node.closest(TRANSCRIPT_TEXTAREA_SELECTOR)) return true;
  return isTranscriptEditorCoreTarget(node);
}

/** waveform scope 快捷键：shell 焦点，或 tier 滚动区内且不在正文编辑（textarea/CM6）。 */
export function isWaveformShortcutContext(
  target: EventTarget | null,
  shell: HTMLElement | null,
  tierScroll: HTMLElement | null,
): boolean {
  const candidates = [target, document.activeElement].filter(Boolean) as Node[];
  for (const node of candidates) {
    if (shell?.contains(node)) return true;
    if (tierScroll?.contains(node) && !isTranscriptTextEditNode(node)) return true;
  }
  return false;
}
// ===== FILE: apps/desktop/src/utils/waveformThemeColors.ts =====
import { COLORS } from "../config/tokens";
import { SEGMENT_FILL_CSS_VAR } from "../config/segmentFillTokens";
import { clearCspLayoutRules, setCspLayoutRules } from "./cspElementLayout";

let colorProbeEl: HTMLDivElement | null = null;

function getColorProbe(): HTMLDivElement | null {
  if (typeof document === "undefined") return null;
  if (!colorProbeEl) {
    colorProbeEl = document.createElement("div");
    colorProbeEl.setAttribute("aria-hidden", "true");
```

```
  colorProbeEl.className = "waveform-color-probe";
  document.documentElement.appendChild(colorProbeEl);
 }
 return colorProbeEl;
}

/** 读取 `:root` 上的 CSS 变量；空值时回退。 */
export function readCssColorVar(name: string, fallback: string): string {
 if (typeof document === "undefined") return fallback;
 const value = getComputedStyle(document.documentElement).getPropertyValue(name).trim();
 return value || fallback;
}

function readAccentActionHex(fallback = "#C58A43"): string {
 return readCssColorVar("--accent-action", fallback);
}

function readAccentActionStrongHex(fallback = "#85530f"): string {
 return readCssColorVar("--accent-action-strong", fallback);
}

/** Canvas fallback — 随当前 accent 解析，避免 Office 主题下仍回退 saffron。 */
function accentMixFallback(actionPct: number, alphaFallback: number): string {
 const accent = readAccentActionHex();
 const expression = `color-mix(in srgb, ${accent} ${actionPct}%, transparent)`;
 const [r, g, b] = hexToRgb(accent) ?? [197, 138, 67];
 return resolveCssColorExpression(expression, `rgba(${r}, ${g}, ${b}, ${alphaFallback})`);
}

/**
 * Waveform selected fallback — same hue as list (`accent → notion-bg`), then alpha so peaks show.
 */
function accentListWashFallback(actionPct: number, listWashAlpha: number): string {
 const accent = readAccentActionHex();
 const bg = readCssColorVar("--notion-bg", "#ffffff");
 const bgHex = bg.startsWith("#") ? bg : "#ffffff";
 const pct = Math.round(listWashAlpha * 100);
 const expression = `color-mix(in srgb, color-mix(in srgb, ${accent} ${actionPct}%, ${bgHex}) ${...
 const [ar, ag, ab] = hexToRgb(accent) ?? [197, 138, 67];
 const [br, bgg, bb] = hexToRgb(bgHex) ?? [255, 255, 255];
 const t = actionPct / 100;
 const r = Math.round(ar * t + br * (1 - t));
 const g = Math.round(ag * t + bgg * (1 - t));
 const b = Math.round(ab * t + bb * (1 - t));
 return resolveCssColorExpression(expression, `rgba(${r}, ${g}, ${b}, ${listWashAlpha})`);
}

function accentStrongMixFallback(strongPct: number, alphaFallback: number): string {
 const strong = readAccentActionStrongHex();
 const expression = `color-mix(in srgb, ${strong} ${strongPct}%, transparent)`;
```

```
  const [r, g, b] = hexToRgb(strong) ?? [133, 83, 15];
  return resolveCssColorExpression(expression, `rgba(${r}, ${g}, ${b}, ${alphaFallback})`);
}

function hexToRgb(hex: string): [number, number, number] | null {
  const normalized = hex.trim().replace(/^#/, "");
  if (normalized.length === 3) {
    const r = parseInt(normalized[0] + normalized[0], 16);
    const g = parseInt(normalized[1] + normalized[1], 16);
    const b = parseInt(normalized[2] + normalized[2], 16);
    return [r, g, b];
  }
  if (normalized.length !== 6) return null;
  const r = parseInt(normalized.slice(0, 2), 16);
  const g = parseInt(normalized.slice(2, 4), 16);
  const b = parseInt(normalized.slice(4, 6), 16);
  if ([r, g, b].some((n) => Number.isNaN(n))) return null;
  return [r, g, b];
}

/**
 * 将含 `var()` / `color-mix()` 的表达式解析为浏览器可绘制的 `rgb(a)`。
 * Canvas 2D 与 WaveSurfer 无法直接使用 color-mix 字符串。
 */
export function resolveCssColorExpression(
  expression: string,
  fallback: string,
  property: "backgroundColor" | "color" = "backgroundColor",
): string {
  if (typeof document === "undefined") return fallback;
  const probe = getColorProbe();
  if (!probe) return fallback;
  setCspLayoutRules(probe, { [property]: expression });
  const resolved = getComputedStyle(probe)[property].trim();
  clearCspLayoutRules(probe);
  const unresolved =
    !resolved ||
    resolved.includes("color-mix(") ||
    resolved.includes("var(") ||
    (resolved === "rgba(0, 0, 0, 0)" && expression.includes("transparent") === false);
  if (unresolved) {
    return fallback;
  }
  return resolved;
}

export type WaveformSurferPalette = {
  waveColor: string;
  progressColor: string;
  cursorColor: string;
```

```
};

/** 读取 tokens.css 中 color-mix 表达式并解析为 canvas 可用的 rgb(a)。 */
function resolveRootFillToken(
  cssVarName: string,
  fallbackExpression: string,
  fallbackRgb: string,
): string {
  const expression = readCssColorVar(cssVarName, fallbackExpression);
  return resolveCssColorExpression(expression, fallbackRgb);
}

/** WaveSurfer peaks 配色 — 对齐 tokens.css `--zen-wf-*`（已播放为中性淡化，不锁 accent）。 */
export function readWaveformSurferPalette(): WaveformSurferPalette {
  const waveColor = readCssColorVar("--zen-wf-wave", COLORS.waveformWave);
  const progressColor = readCssColorVar(
    "--zen-wf-progress-played",
    COLORS.waveformProgressPlayed,
  );
  const cursorColor = readCssColorVar("--zen-wf-cursor", COLORS.waveformCursor);
  return { waveColor, progressColor, cursorColor };
}

export type WaveformSegmentBandPalette = {
  selected: string;
  inSelection: string;
  lowConfidence: string;
  visited: string;
  idle: string;
  border: string;
  selectedBorder: string;
  inSelectionBorder: string;
};

/** 语段 band canvas 配色 — 与 segmentChrome / tokens.css `--segment-fill-*` 同源。 */
let cachedSegmentBandPalette: WaveformSegmentBandPalette | null = null;

export function invalidateWaveformSegmentBandPaletteCache(): void {
  cachedSegmentBandPalette = null;
}

export function readWaveformSegmentBandPalette(): WaveformSegmentBandPalette {
  if (cachedSegmentBandPalette) return cachedSegmentBandPalette;
  cachedSegmentBandPalette = {
    selected: resolveRootFillToken(
      SEGMENT_FILL_CSS_VAR.selected,
      "color-mix(in srgb, color-mix(in srgb, var(--accent-action) 12%, var(--notion-bg)) 82%, tra...
      accentListWashFallback(12, 0.82),
    ),
    inSelection: resolveRootFillToken(
```

```
    SEGMENT_FILL_CSS_VAR.inSelectionWaveform,
    "color-mix(in srgb, var(--accent-action) 8%, transparent)",
    accentMixFallback(8, 0.08),
   ),
   lowConfidence: resolveRootFillToken(
    SEGMENT_FILL_CSS_VAR.lowConfidence,
    "color-mix(in srgb, var(--notion-text-light) 10%, transparent)",
    "rgba(156, 163, 175, 0.10)",
   ),
   visited: resolveRootFillToken(
    SEGMENT_FILL_CSS_VAR.visited,
    "color-mix(in srgb, var(--notion-text) 10%, transparent)",
    "rgba(55, 53, 47, 0.10)",
   ),
   idle: resolveRootFillToken(
    SEGMENT_FILL_CSS_VAR.idle,
    "color-mix(in srgb, var(--notion-text) 5%, transparent)",
    "rgba(55, 53, 47, 0.05)",
   ),
   border: resolveRootFillToken(
    SEGMENT_FILL_CSS_VAR.border,
    "color-mix(in srgb, var(--notion-text) 24%, transparent)",
    "rgba(55, 53, 47, 0.24)",
   ),
   selectedBorder: resolveRootFillToken(
    SEGMENT_FILL_CSS_VAR.selectedBorder,
    "color-mix(in srgb, var(--accent-action-strong) 48%, transparent)",
    accentStrongMixFallback(48, 0.48),
   ),
   inSelectionBorder: resolveRootFillToken(
    SEGMENT_FILL_CSS_VAR.inSelectionBorder,
    "color-mix(in srgb, var(--accent-action) 34%, transparent)",
    accentMixFallback(34, 0.34),
   ),
  };
  return cachedSegmentBandPalette;
}
// ===== FILE: apps/desktop/src/utils/waveformTimelineMetrics.ts =====
import { clampPxPerSecForLayout, computeTimelineWidthPx } from "./pxPerSec";

export type ResolveWaveformTimelineMetricsInput = {
  /** WaveSurfer / media element duration once known. */
  wsDurationSec: number;
  /** Cached peaks manifest duration (fallback before WS ready). */
  peaksStatusDurationSec: number;
  pxPerSec: number;
};

export type WaveformTimelineMetrics = {
  /** Layout / segments / scroll / minimap duration truth. */
```

```
 mediaDurationSec: number;
 timelineWidthPx: number;
};

/** Merge WS and peaks-manifest durations into one layout timeline length. */
export function resolveMediaDurationSec(input: {
 wsDurationSec: number;
 peaksStatusDurationSec: number;
}): number {
 const d = input.wsDurationSec || input.peaksStatusDurationSec || 0;
 return d > 0 ? d : 0;
}

/**
 * Single layout duration for seek / peaks load / scroll clamp.
 * Synced layout ref wins; otherwise merged WS + peaks manifest (+ optional cache).
 */
export function resolveLayoutDurationSec(input: {
 layoutDurationSecRef?: number;
 layoutDurationSec?: number;
 wsDurationSec?: number;
 peaksStatusDurationSec?: number;
 peakCacheDurationSec?: number;
}): number {
 const fromRef = input.layoutDurationSecRef ?? 0;
 if (fromRef > 0) return fromRef;
 const fromProp = input.layoutDurationSec ?? 0;
 if (fromProp > 0) return fromProp;
 const peaksManifest = input.peaksStatusDurationSec ?? 0;
 const peaksCache = input.peakCacheDurationSec ?? 0;
 return resolveMediaDurationSec({
   wsDurationSec: input.wsDurationSec ?? 0,
   peaksStatusDurationSec: peaksManifest > 0 ? peaksManifest : peaksCache,
 });
}

/** Single export for timeline width and duration helpers. */
export function resolveWaveformTimelineMetrics(
 input: ResolveWaveformTimelineMetricsInput,
): WaveformTimelineMetrics {
 const mediaDurationSec = resolveMediaDurationSec({
   wsDurationSec: input.wsDurationSec,
   peaksStatusDurationSec: input.peaksStatusDurationSec,
 });
 const layoutPxPerSec =
   mediaDurationSec > 0
     ? clampPxPerSecForLayout(input.pxPerSec, mediaDurationSec)
     : input.pxPerSec;
 const timelineWidthPx = computeTimelineWidthPx(mediaDurationSec, layoutPxPerSec);
```

```
  return {
    mediaDurationSec,
    timelineWidthPx,
  };
}
// ===== FILE: apps/desktop/src/utils/waveformViewMode.ts =====
/** 语段选中来源（用于行为分支）。 */
export type SegmentSelectSource =
  | "list"
  | "listAdvance"
  | "listKeyboard"
  | "waveform"
  | "waveformKeyboard"
  /** 右键菜单：不 reveal/seek；列表 scroll 仍随 primary。 */
  | "contextMenu"
  /** 列表 range / 波形 lasso 多选：不 reveal/seek；列表 scroll + pin。 */
  | "multiSelect";

export type SegmentSelectAtOptions = {
  shiftKey?: boolean;
  toggle?: boolean;
  /** Waveform pointerdown session that already handled preview seek/reveal. */
  previewSessionId?: string;
  /**
   * Playing-defer path: pointerdown already updated CM6 primary, so chrome-match
   * would skip seek/reveal — force both on pointerup (Transport Authority).
   */
  forceSeek?: boolean;
  /** listKeyboard burst: SC2 + scroll only; SC1 commit on keyup. */
  burst?: boolean;
  /** Caller focuses segment textarea; skip waveform shell focus (e.g. after drag-create). */
  preferSegmentTextFocus?: boolean;
  /** Explicit scoped-play owns start; do not queue sticky listen-jump playback. */
  suppressStickyListenJump?: boolean;
};

export function isListKeyboardBurstStep(
  source: SegmentSelectSource,
  opts?: SegmentSelectAtOptions,
): boolean {
  return source === "listKeyboard" && opts?.burst === true && !opts?.shiftKey && !opts?.toggle;
}

/** Waveform arrow keys: SC2 + seek only; SC1 + tier reveal on keyup. */
export function isWaveformKeyboardBurstStep(
  source: SegmentSelectSource,
  opts?: SegmentSelectAtOptions,
): boolean {
  return source === "waveformKeyboard" && !opts?.shiftKey && !opts?.toggle;
}
```

```
/** 列表/文本编辑选中时不抢 textarea 焦点。 */
export function shouldFocusWaveformShellForSelectSource(source: SegmentSelectSource): boolean {
 return source === "waveform" || source === "waveformKeyboard";
}

/** 仅 waveform 单选：列表行已在视口内时跳过 scroll/pin（SCB-2）。 */
export function shouldSkipListScrollWhenInViewport(source: SegmentSelectSource): boolean {
 return source === "waveform" || source === "waveformKeyboard";
}

/** Store + React subscribers — skip imperative overlay CSP on hot select paths. */
export function shouldSkipImperativeSelectionChrome(source: SegmentSelectSource): boolean {
 return (
  source === "waveform" ||
  source === "waveformKeyboard" ||
  source === "list" ||
  source === "listAdvance" ||
  source === "listKeyboard" ||
  source === "contextMenu" ||
  source === "multiSelect"
 );
}
// ===== FILE: apps/desktop/src/utils/waveformViewport.ts =====
import type { RefObject } from "react";
import { logRuntimeParity } from "../services/runtimeParity";
import {
 isTierScrollFrameActive,
 readPlaybackFractionalPx,
 readTierViewportMetricsDuringScrollFrame,
} from "./tierScrollFrameCoordinator";
import { PLAYBACK_SUBPIXEL_ENABLED } from "./waveformPlaybackSubpixel";
import { setCspLayoutRules } from "./cspElementLayout";

export type TierScrollLiveRefs = {
 scrollLeftRef: RefObject<number>;
 clientWidthRef: RefObject<number>;
};

export type TierScrollLayoutMetrics = {
 scrollLeftPx: number;
 clientWidthPx: number;
};

/** Tier scrollport width — set imperatively on the tier element for sticky waveform clip. */
export const WAVEFORM_TIER_VIEWPORT_WIDTH_VAR = "--waveform-tier-viewport-width";

export const WAVEFORM_TIER_VIEWPORT_WIDTH_CLASS = "waveform-tier-viewport-width";

/** Segment chrome height during drag/preview — match visual shell, not deferred painted height. */
```

```
export function resolveWaveformSegmentLayoutHeightPx(
  visualHeightPx: number,
  paintedHeightPx: number,
  previewActive: boolean,
): number {
  const visual = Math.max(1, visualHeightPx);
  const painted = Math.max(1, paintedHeightPx);
  return previewActive ? visual : painted;
}

/** Stretch painted waveform to visual shell height while WaveSurfer redraw is pending. */
export function resolveWaveformVerticalScalePreview(
  visualHeightPx: number,
  paintedHeightPx: number,
): { scale: number; active: boolean; transform: string | undefined } {
  const painted = Math.max(1, paintedHeightPx);
  const visual = Math.max(1, visualHeightPx);
  const scale = visual / painted;
  const active = Math.abs(scale - 1) > 0.001;
  return {
    scale,
    active,
    transform: active ? `scaleY(${scale})` : undefined,
  };
}

export function writeWaveformTierViewportWidthVar(
  tierEl: HTMLElement,
  viewportWidthPx: number,
): void {
  if (viewportWidthPx > 0) {
    setCspLayoutRules(tierEl, {
      [WAVEFORM_TIER_VIEWPORT_WIDTH_VAR]: `${viewportWidthPx}px`,
    });
  }
}

/** Read the widest known tier viewport width (live ref > DOM > committed layout). */
export function resolveTierViewportWidthPx(input: {
  tierScrollEl?: HTMLElement | null;
  layoutClientWidthPx?: number;
  liveClientWidthPx?: number;
}): number {
  const live = input.liveClientWidthPx ?? 0;
  const tier = input.tierScrollEl?.clientWidth ?? 0;
  const layout = input.layoutClientWidthPx ?? 0;
  return Math.max(live, tier, layout);
}

/** Prefer live tier DOM scroll; fall back to live ref then committed layout. */
```

```
export function resolveTierScrollLeftPx(input: {
 tierScrollEl?: HTMLElement | null;
 layoutScrollLeftPx: number;
 liveScrollLeftRef?: RefObject<number>;
}): number {
 const domScrollLeft = input.tierScrollEl?.scrollLeft;
 if (domScrollLeft != null && Number.isFinite(domScrollLeft)) {
  return domScrollLeft;
 }
 const live = input.liveScrollLeftRef?.current;
 if (live != null && Number.isFinite(live)) return live;
 return input.layoutScrollLeftPx;
}

/** Single read path for overlay / minimap / playback chrome (TRUTH-006/009/023). */
export function resolveTierViewportMetrics(input: {
 tierScrollEl?: HTMLElement | null;
 tierScrollLive?: TierScrollLiveRefs;
 tierScrollLayout: TierScrollLayoutMetrics;
}): { scrollLeftPx: number; viewportWidthPx: number } {
 const tier = input.tierScrollEl;
 return {
  scrollLeftPx: resolveTierScrollLeftPx({
   tierScrollEl: tier,
   layoutScrollLeftPx: input.tierScrollLayout.scrollLeftPx,
   liveScrollLeftRef: input.tierScrollLive?.scrollLeftRef,
  }),
  viewportWidthPx: Math.max(
   1,
   resolveTierViewportWidthPx({
    tierScrollEl: tier,
    layoutClientWidthPx: input.tierScrollLayout.clientWidthPx,
    liveClientWidthPx: input.tierScrollLive?.clientWidthRef.current,
   }),
  ),
 };
}

/** Scroll-frame subscribers: prefer per-frame snapshot; fall back to DOM-first read. */
let devWarnedMissingScrollFrameSnapshot = false;

/**
 * `scrollLeftPx` stays the integer, browser-quantized native scroll (S) — feed it to
 * window/slice computation so `computeWaveformViewportPeaksWindow` never rewindows on
 * sub-pixel jitter. `effectiveScrollLeftPx = S + fraction` (T) is the full-precision
 * horizontal truth for time→viewport readers (playhead) — center-follow subpixel (A2).
 */
export function resolveTierViewportMetricsDuringScrollFrame(input: {
 tierScrollEl?: HTMLElement | null;
 tierScrollLive?: TierScrollLiveRefs;
```

```
  tierScrollLayout: TierScrollLayoutMetrics;
}): {
  scrollLeftPx: number;
  viewportWidthPx: number;
  effectiveScrollLeftPx: number;
  fraction: number;
} {
  const base = resolveTierViewportMetricsDuringScrollFrameBase(input);
  const fraction = PLAYBACK_SUBPIXEL_ENABLED ? readPlaybackFractionalPx() : 0;
  return {
    scrollLeftPx: base.scrollLeftPx,
    viewportWidthPx: base.viewportWidthPx,
    effectiveScrollLeftPx: base.scrollLeftPx + fraction,
    fraction,
  };
}

function resolveTierViewportMetricsDuringScrollFrameBase(input: {
  tierScrollEl?: HTMLElement | null;
  tierScrollLive?: TierScrollLiveRefs;
  tierScrollLayout: TierScrollLayoutMetrics;
}): { scrollLeftPx: number; viewportWidthPx: number } {
  const snapshot = readTierViewportMetricsDuringScrollFrame();
  if (snapshot) return snapshot;
  if (isTierScrollFrameActive() && !devWarnedMissingScrollFrameSnapshot) {
    devWarnedMissingScrollFrameSnapshot = true;
    logRuntimeParity(
      "waveform",
      "scroll-frame subscriber read metrics without snapshot; falling back to DOM",
      "WARN",
    );
  }
  return resolveTierViewportMetrics(input);
}
// ===== FILE: apps/desktop/src/utils/waveformViewportStretch.ts =====
import { setCspLayoutRules, clearCspLayoutRules } from "./cspElementLayout";

const STRETCH_EPSILON = 0.001;

export type WriteWaveformShellLayoutInput = {
  timelineShell?: HTMLElement | null;
  peaksStageShell?: HTMLElement | null;
  stickyShell?: HTMLElement | null;
  timelineWidthPx: number;
  viewportWidthPx: number;
};

/** Imperatively sync timeline / stage / sticky shells (layout truth for resize + zoom). */
export function writeWaveformShellLayout(input: WriteWaveformShellLayoutInput): void {
  const {
```

```
    timelineShell,
    peaksStageShell,
    stickyShell,
    timelineWidthPx,
    viewportWidthPx,
  } = input;
  if (timelineWidthPx <= 0 || viewportWidthPx <= 0) return;
  const stageWidthPx = Math.max(timelineWidthPx, viewportWidthPx);
  if (timelineShell) writeWaveformTimelineShellWidth(timelineShell, timelineWidthPx);
  if (peaksStageShell) writeWaveformPeaksStageShellWidth(peaksStageShell, stageWidthPx);
  if (stickyShell) writeWaveformStickyShellWidth(stickyShell, viewportWidthPx);
}

export function writeWaveformStickyShellWidth(
  stickyEl: HTMLElement,
  viewportWidthPx: number,
): void {
  if (viewportWidthPx <= 0) return;
  setCspLayoutRules(stickyEl, { width: viewportWidthPx });
}

export function writeWaveformTimelineShellWidth(
  shellEl: HTMLElement,
  timelineWidthPx: number,
): void {
  if (timelineWidthPx <= 0) return;
  setCspLayoutRules(shellEl, { width: timelineWidthPx });
}

export function writeWaveformPeaksStageShellWidth(
  shellEl: HTMLElement,
  stageWidthPx: number,
): void {
  if (stageWidthPx <= 0) return;
  setCspLayoutRules(shellEl, { width: stageWidthPx });
}

export function applyWaveformViewportStretch(stretchEl: HTMLElement, ratio: number): void {
  if (!Number.isFinite(ratio) || Math.abs(ratio - 1) <= STRETCH_EPSILON) {
    clearWaveformViewportStretch(stretchEl);
    return;
  }
  setCspLayoutRules(stretchEl, {
    transformOrigin: "left top",
    transform: `scaleX(${ratio})`,
  });
}

export function clearWaveformViewportStretch(stretchEl: HTMLElement | null | undefined): void {
  if (!stretchEl) return;
```

```
  clearCspLayoutRules(stretchEl);
}

export function computeViewportStretchRatio(
  previousWidthPx: number,
  nextWidthPx: number,
): number | null {
  if (previousWidthPx <= 0 || nextWidthPx <= 0) return null;
  if (Math.abs(previousWidthPx - nextWidthPx) <= 1) return null;
  return nextWidthPx / previousWidthPx;
}
// ===== FILE: apps/desktop/src/utils/waveformZoomBarState.ts =====
import {
  computeFitAllPxPerSec,
  computeTimelineWidthPx,
  FIT_ALL_FILL_GAP_MIN_PX,
  FIT_ALL_FILL_GAP_RATIO,
  isFitAllTimelineFilledInViewport,
  isTimelineFitInViewport,
  PX_PER_SEC_MAX,
  PX_PER_SEC_PEAKS_QUANTUM,
  resolveDefaultEditingPxPerSec,
  resolveQuantizedFitSelectionPxPerSec,
  TIMELINE_PX_PER_SEC,
  type WaveformZoomLayoutIntent,
  type WaveformZoomSliderRange,
} from "./pxPerSec";

const ZOOM_EPS = 0.001;

/** px/s may sit slightly below computed fit-all min (binary search / rounding); don't treat as o...
function sliderMinTolerancePx(sliderMinPx: number): number {
  return Math.max(ZOOM_EPS, Math.min(PX_PER_SEC_PEAKS_QUANTUM, sliderMinPx * 0.05));
}

export function isPxPerSecBelowSliderMin(pxPerSec: number, sliderMinPx: number): boolean {
  return pxPerSec < sliderMinPx - sliderMinTolerancePx(sliderMinPx);
}

export function isPxPerSecNearFitAll(
  pxPerSec: number,
  fitAllPxPerSec: number,
): boolean {
  const tol = sliderMinTolerancePx(fitAllPxPerSec);
  return (
    Math.abs(pxPerSec - fitAllPxPerSec) <= Math.max(tol, fitAllPxPerSec * 0.05)
  );
}

/** True when px/s was fit-all for the timeline width currently rendered. */
```

```
function wasFitAllPxPerSecForTimelineWidth(input: {
  timelineWidthPx: number;
  durationSec: number;
  pxPerSec: number;
}): boolean {
  const { timelineWidthPx, durationSec, pxPerSec } = input;
  if (!(timelineWidthPx > 0 && durationSec > 0 && Number.isFinite(pxPerSec))) {
    return false;
  }
  const fitAllForWidth = computeFitAllPxPerSec(timelineWidthPx, durationSec);
  const tol = sliderMinTolerancePx(fitAllForWidth);
  return Math.abs(pxPerSec - fitAllForWidth) <= Math.max(tol, fitAllForWidth * 0.05);
}

/**
 * When the user is at (or near) fit-all, recompute px/s after viewport or duration
 * changes so the timeline fills the tier instead of leaving blank space on the right.
 */
export function resolveFitAllPxPerSecAdjustment(
  viewportWidthPx: number,
  durationSec: number,
  currentPxPerSec: number,
  options?: {
    staleFitAllOnViewportGrow?: boolean;
    layoutIntent?: WaveformZoomLayoutIntent;
  },
): number | null {
  if (viewportWidthPx <= 0 || durationSec <= 0 || !Number.isFinite(currentPxPerSec)) {
    return null;
  }
  const fitAll = computeFitAllPxPerSec(viewportWidthPx, durationSec);
  const tol = sliderMinTolerancePx(fitAll);
  const timelineW = computeTimelineWidthPx(durationSec, currentPxPerSec);
  const fillGapPx = viewportWidthPx - timelineW;
  const fillGapThreshold = Math.max(
    FIT_ALL_FILL_GAP_MIN_PX,
    viewportWidthPx * FIT_ALL_FILL_GAP_RATIO,
  );

  if (options?.layoutIntent === "fit-all") {
    if (!isFitAllTimelineFilledInViewport(viewportWidthPx, durationSec, currentPxPerSec)) {
      return fitAll;
    }
    return null;
  }

  if (
    options?.staleFitAllOnViewportGrow &&
    fillGapPx > fillGapThreshold &&
    wasFitAllPxPerSecForTimelineWidth({
```

```
    timelineWidthPx: timelineW,
    durationSec,
    pxPerSec: currentPxPerSec,
  })
 ) {
  return fitAll;
 }

 const nearFitAll =
  currentPxPerSec >= fitAll - tol &&
  Math.abs(currentPxPerSec - fitAll) <= Math.max(tol, fitAll * 0.05);
 if (!nearFitAll) {
  return null;
 }
 if (fillGapPx > fillGapThreshold) {
  return fitAll;
 }
 if (
  !isTimelineFitInViewport(viewportWidthPx, durationSec, currentPxPerSec) &&
  currentPxPerSec <= fitAll * 1.02
 ) {
  return fitAll;
 }
 return null;
}

/** 当前横向缩放「视图模式」（由 px/s + 视口 + 时长 + 选中语段派生，非持久偏好）。 */
type WaveformZoomViewMode = "fit-selection" | "default" | "custom";

export type WaveformZoomBarUiInput = {
 pxPerSec: number;
 viewportWidthPx: number;
 durationSec: number;
 layoutIntent?: WaveformZoomLayoutIntent;
 /** 有选中语段时传入，用于判定 fit-selection 视图。 */
 selectedStartSec?: number;
 selectedEndSec?: number;
 /** Packable spans for long-media median default highlight. */
 segmentSpansSec?: ReadonlyArray<number>;
 sliderRange?: WaveformZoomSliderRange;
};

/** 缩放条三枚模式按钮的互斥激活态（至多一项）。 */
export type WaveformZoomBarActiveMode = "fit-all" | "fit-selection" | "default" | null;

export type WaveformZoomBarUiState = {
 viewMode: WaveformZoomViewMode;
 activeMode: WaveformZoomBarActiveMode;
 atDefaultZoom: boolean;
 atMinZoom: boolean;
```

```
  atMaxZoom: boolean;
  atFitSelectionZoom: boolean;
  /** px/s 低于当前文件手动下限（如语段 fit 到极长句）。 */
  belowManualSliderRange: boolean;
  atFitAllZoom: boolean;
  zoomPercentLabel: number;
};

function resolveEditingDefaultPxPerSec(input: WaveformZoomBarUiInput): number {
  if (input.viewportWidthPx > 0 && input.durationSec >= 0.5) {
    return resolveDefaultEditingPxPerSec(input.viewportWidthPx, input.durationSec, {
      segmentSpansSec: input.segmentSpansSec,
    });
  }
  return TIMELINE_PX_PER_SEC;
}

function isNearEditingDefaultPxPerSec(pxPerSec: number, defaultPx: number): boolean {
  const tol = Math.max(ZOOM_EPS, defaultPx * 0.01);
  return Math.abs(pxPerSec - defaultPx) <= tol;
}

function isNearQuantizedFitSelectionPxPerSec(
  pxPerSec: number,
  viewportWidthPx: number,
  selectedStartSec?: number,
  selectedEndSec?: number,
): boolean {
  if (
    viewportWidthPx <= 0 ||
    selectedStartSec == null ||
    selectedEndSec == null ||
    !Number.isFinite(selectedStartSec) ||
    !Number.isFinite(selectedEndSec)
  ) {
    return false;
  }
  const target = resolveQuantizedFitSelectionPxPerSec(
    viewportWidthPx,
    selectedStartSec,
    selectedEndSec,
  );
  const tol = Math.max(ZOOM_EPS, target * 0.01);
  return Math.abs(pxPerSec - target) <= tol;
}

/** layoutIntent 优先；manual 时不亮模式键；无 intent 时按 px 接近度回退。 */
export function resolveWaveformZoomBarActiveMode(
  input: WaveformZoomBarUiInput,
): WaveformZoomBarActiveMode {
```

```
  const { pxPerSec, layoutIntent, viewportWidthPx, durationSec, selectedStartSec, selectedEndSec } =
    input;

  if (layoutIntent === "manual") {
    return null;
  }
  if (layoutIntent === "fit-all") {
    return "fit-all";
  }
  if (layoutIntent === "fit-selection") {
    return "fit-selection";
  }
  if (layoutIntent === "default") {
    return "default";
  }

  if (viewportWidthPx <= 0 || durationSec < 0.5) {
    if (Math.abs(pxPerSec - TIMELINE_PX_PER_SEC) < ZOOM_EPS) {
      return "default";
    }
    return null;
  }

  if (isFitAllTimelineFilledInViewport(viewportWidthPx, durationSec, pxPerSec)) {
    return "fit-all";
  }
  const defaultPx = resolveDefaultEditingPxPerSec(viewportWidthPx, durationSec);
  if (isNearEditingDefaultPxPerSec(pxPerSec, defaultPx)) {
    return "default";
  }
  if (isNearQuantizedFitSelectionPxPerSec(
    pxPerSec,
    viewportWidthPx,
    selectedStartSec,
    selectedEndSec,
  )) {
    return "fit-selection";
  }
  return null;
}

export function isNearEditingDefaultForMedia(
  pxPerSec: number,
  viewportWidthPx: number,
  durationSec: number,
  options?: { segmentSpansSec?: ReadonlyArray<number> },
): boolean {
  return isNearEditingDefaultPxPerSec(
    pxPerSec,
    resolveEditingDefaultPxPerSec({
```

```
    pxPerSec,
    viewportWidthPx,
    durationSec,
    segmentSpansSec: options?.segmentSpansSec,
   }),
  );
}

function deriveWaveformZoomViewMode(input: WaveformZoomBarUiInput): WaveformZoomViewMode {
  const activeMode = resolveWaveformZoomBarActiveMode(input);
  if (activeMode === "default") {
    return "default";
  }
  if (activeMode === "fit-selection") {
    return "fit-selection";
  }
  return "custom";
}

/** 缩放条：视图模式 + 离散按钮派生 UI（纯函数）。 */
export function computeWaveformZoomBarUiState(input: WaveformZoomBarUiInput | number): WaveformZo...
  const resolved: WaveformZoomBarUiInput =
    typeof input === "number"
      ? { pxPerSec: input, viewportWidthPx: 0, durationSec: 0 }
      : input;

  const {
    pxPerSec,
    viewportWidthPx,
    durationSec,
    sliderRange,
  } = resolved;
  const defaultPx = resolveEditingDefaultPxPerSec(resolved);
  const activeMode = resolveWaveformZoomBarActiveMode(resolved);
  const atFitAllZoom = activeMode === "fit-all";
  const atFitSelectionZoom = activeMode === "fit-selection";
  const atDefaultZoom = activeMode === "default";
  const sliderMinPx =
    sliderRange?.minPxPerSec ??
    (viewportWidthPx > 0 && durationSec > 0
      ? computeFitAllPxPerSec(viewportWidthPx, durationSec)
      : pxPerSec);
  const belowManualSliderRange = isPxPerSecBelowSliderMin(pxPerSec, sliderMinPx);
  const atMinFitAllFill =
    viewportWidthPx > 0 &&
    durationSec > 0 &&
    isFitAllTimelineFilledInViewport(viewportWidthPx, durationSec, pxPerSec);

  return {
    viewMode: deriveWaveformZoomViewMode(resolved),
```

```
    activeMode,
    atDefaultZoom,
    atMinZoom: belowManualSliderRange
      ? false
      : Math.abs(pxPerSec - sliderMinPx) < ZOOM_EPS || atMinFitAllFill,
    atMaxZoom: pxPerSec >= (sliderRange?.maxPxPerSec ?? PX_PER_SEC_MAX) - ZOOM_EPS,
    atFitSelectionZoom,
    belowManualSliderRange,
    atFitAllZoom,
    zoomPercentLabel: Math.round((pxPerSec / defaultPx) * 100),
  };
}
// ===== FILE: apps/desktop/src/utils/waveformZoomBarTitles.ts =====
import { FIT_SELECTION_VIEWPORT_RATIO } from "./pxPerSec";

export function fitSelectionPercentLabel(): string {
  return `${Math.round(FIT_SELECTION_VIEWPORT_RATIO * 100)}%`;
}

export function resolveFitSelectionTitle(hasSelection: boolean, active: boolean): string {
  if (!hasSelection) {
    return "请先在语段列表或波形上选中一条语段";
  }
  const pct = fitSelectionPercentLabel();
  if (active) {
    return `语段适配（已激活）：选中语段约占视口 ${pct} 宽，并已滚入可见区域`;
  }
  return `语段适配：将选中语段缩放到约占视口 ${pct} 宽，并滚入可见区域`;
}

export function resolveFitAllTitle(active: boolean): string {
  if (active) {
    return "整段可见（已激活）：整段音频时间轴已贴满视口宽度";
  }
  return "整段可见：缩小至整段音频贴满视口宽度（本文件最宽可见）";
}

export function resolveZoomOutTitle(atMinZoom: boolean): string {
  if (atMinZoom) {
    return "已为本文件最宽可见（整段贴满视口）";
  }
  return "缩小一级";
}

export function resolveZoomInTitle(atMaxZoom: boolean): string {
  if (atMaxZoom) {
    return "已为本文件最大缩放";
  }
  return "放大一级";
}
```

```
export function resolveResetTitle(active: boolean): string {
 if (active) {
  return "默认比例（已激活）：当前为本文件编辑默认缩放（标签约 100%）";
 }
 return "默认比例：恢复本文件编辑默认（几何中点；±5 步可达最宽/最大）";
}

export function hasWaveformSegmentSelection(
 selectedStartSec?: number,
 selectedEndSec?: number,
): boolean {
 return (
  selectedStartSec != null &&
  selectedEndSec != null &&
  Number.isFinite(selectedStartSec) &&
  Number.isFinite(selectedEndSec)
 );
}
// ===== FILE: apps/desktop/src/utils/waveformZoomScroll.ts =====
import { computeTimelineWidthPx } from "./segmentLayout";
import { clampTimelineScrollLeftPx } from "./waveformScrollSync";
import { scrollPxPreservingViewportCenterTime } from "./waveformProjection";

/** Remap tier scroll when px/s changes so the viewport center time stays fixed. */
export function remapWaveformScrollLeftPx(input: {
 scrollLeftPx: number;
 oldPxPerSec: number;
 newPxPerSec: number;
 durationSec: number;
 viewportWidthPx: number;
}): number {
 const oldTw = computeTimelineWidthPx(input.durationSec, input.oldPxPerSec);
 const newTw = computeTimelineWidthPx(input.durationSec, input.newPxPerSec);
 return scrollPxPreservingViewportCenterTime({
  scrollLeftPx: input.scrollLeftPx,
  oldTimelineWidthPx: oldTw,
  newTimelineWidthPx: newTw,
  durationSec: input.durationSec,
  viewportWidthPx: input.viewportWidthPx,
 });
}

/** Clamp scroll to the current timeline width. */
export function clampWaveformScrollLeftPx(input: {
 scrollLeftPx: number;
 pxPerSec: number;
 durationSec: number;
 viewportWidthPx: number;
}): number {
```

```
  const tw = computeTimelineWidthPx(input.durationSec, input.pxPerSec);
  return clampTimelineScrollLeftPx({
    scrollLeftPx: input.scrollLeftPx,
    timelineWidthPx: tw,
    viewportWidthPx: input.viewportWidthPx,
  });
}
// ===== FILE: apps/desktop/src/utils/waveformZoomSlider.ts =====
import type { WaveformZoomSliderRange } from "./pxPerSec";
import { clampPxPerSecInSliderRange, resolveWaveformZoomStepRatio } from "./pxPerSec";
import { isPxPerSecBelowSliderMin } from "./waveformZoomBarState";

/** +/- 与键盘 zoom：显著低于滑块下限时 snap 到 min，否则按文件区间对数步进。 */
export function computeZoomInPxPerSec(
  pxPerSec: number,
  range: WaveformZoomSliderRange,
): number {
  if (isPxPerSecBelowSliderMin(pxPerSec, range.minPxPerSec)) {
    return range.minPxPerSec;
  }
  const ratio = resolveWaveformZoomStepRatio(range);
  return clampPxPerSecInSliderRange(pxPerSec * ratio, range);
}

export function computeZoomOutPxPerSec(
  pxPerSec: number,
  range: WaveformZoomSliderRange,
): number {
  if (isPxPerSecBelowSliderMin(pxPerSec, range.minPxPerSec)) {
    return range.minPxPerSec;
  }
  const ratio = resolveWaveformZoomStepRatio(range);
  return clampPxPerSecInSliderRange(pxPerSec / ratio, range);
}

export function pxPerSecToSliderPos(pxPerSec: number, range: WaveformZoomSliderRange): number {
  const { minPxPerSec: lo, maxPxPerSec: hi } = range;
  const c = clampPxPerSecInSliderRange(pxPerSec, range);
  if (hi <= lo) return 0;
  return Math.round((Math.log(c / lo) / Math.log(hi / lo)) * 1000);
}

export function sliderPosToPxPerSec(pos: number, range: WaveformZoomSliderRange): number {
  const { minPxPerSec: lo, maxPxPerSec: hi } = range;
  const t = Math.min(1000, Math.max(0, pos));
  if (hi <= lo) return lo;
  return clampPxPerSecInSliderRange(lo * (hi / lo) ** (t / 1000), range);
}
// ===== FILE: apps/desktop/src/utils/zhPinyinCompare.ts =====
function createZhPinyinCollator(): Intl.Collator {
```

```
 try {
  return new Intl.Collator("zh-Hans-CN-u-co-pinyin", { numeric: true });
 } catch {
  return new Intl.Collator("zh-Hans-CN", { numeric: true });
 }
}

const zhPinyinCollator = createZhPinyinCollator();

/** 中文按拼音序比较（用于列表排序）。 */
export function compareZhPinyin(a: string, b: string): number {
 return zhPinyinCollator.compare(a, b);
}
// ===== FILE: apps/desktop/src/App.tsx =====
import { ProjectPanel } from "./components/ProjectPanel";
import { ToastHost } from "./components/ToastHost";
import { BundledAsrModelsSeedOverlay } from "./components/BundledAsrModelsSeedOverlay";
import { AppUpdateConfirmDialog } from "./components/AppUpdateConfirmDialog";
import { useBundledAsrModelsSeed } from "./hooks/useBundledAsrModelsSeed";
import { useAppUpdateCheckOnLaunch } from "./hooks/useAppUpdateCheckOnLaunch";
import "./App.css";

export default function App() {
 const bundledSeed = useBundledAsrModelsSeed();
 const launchUpdate = useAppUpdateCheckOnLaunch();

 return (
  <>
   {!bundledSeed.blocking ? (
    <main className="shell">
     <div className="shell-body">
      <ProjectPanel />
     </div>
    </main>
   ) : null}
   <BundledAsrModelsSeedOverlay
    gate={bundledSeed.gate}
    onRetry={() => void bundledSeed.retrySeed()}
   />
   <ToastHost />
   <AppUpdateConfirmDialog
    open={launchUpdate.dialogOpen}
    busy={launchUpdate.dialogBusy}
    version={launchUpdate.dialogVersion}
    notes={launchUpdate.dialogNotes}
    onCancel={launchUpdate.onDialogCancel}
    onConfirm={() => void launchUpdate.onDialogConfirm()}
   />
  </>
 );
```

```
}
// ===== FILE: apps/desktop/src/main.tsx =====
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { AppErrorBoundary } from "./components/AppErrorBoundary";
import { isTauriRuntime } from "./config/env";
import { ensureAppWindowCloseGuardRegistered } from "./services/appWindowCloseGuard";
import { logRuntimeParityBootstrap } from "./services/runtimeParityBootstrap";
import { bootstrapShellCapabilities } from "./services/shellCapabilities";
import { installWaveformZoomProfileDevTools } from "./services/waveform/waveformZoomProfile";
import { installWaveformScrollProfileDevTools } from "./services/waveform/waveformScrollProfileDe...
import { installSegmentProbeDevTools } from "./services/waveform/segmentProbeDevTools";
import { installSelectionLatencyProfileDevTools } from "./services/ui/selectionLatencyProfile";
import { initOfficeShellTheme } from "./services/ui/officeShellTheme";
import { initUiDisplayScale } from "./services/ui/uiDisplayScale";
import { bootstrapCspStyleNonce } from "./utils/cspNonceStyleRegistry";
import { logReleaseFrontendProbe } from "./utils/releaseFrontendProbe";
import "./zen-tailwind.css";

initOfficeShellTheme();
initUiDisplayScale();

ensureAppWindowCloseGuardRegistered();
installWaveformZoomProfileDevTools();
installWaveformScrollProfileDevTools();
installSegmentProbeDevTools();
installSelectionLatencyProfileDevTools();

const rootEl = document.getElementById("root");
if (!rootEl) {
 throw new Error('Missing root element with id "root"');
}

async function mountApp(root: HTMLElement): Promise<void> {
 if (isTauriRuntime()) {
  await bootstrapCspStyleNonce();
  logReleaseFrontendProbe();
  await bootstrapShellCapabilities();
  await logRuntimeParityBootstrap();
 }
 ReactDOM.createRoot(root).render(
  <React.StrictMode>
   <AppErrorBoundary>
    <App />
   </AppErrorBoundary>
  </React.StrictMode>,
 );
}
```

```
void mountApp(rootEl);
```